



Ben-Gurion University of the
Negev
Faculty of Engineering Sciences
Department of Mechanical
Engineering



Final Project

Graduate Course in Mechatronic Systems
Nonlinear Control and Real-Time Digital Implementation

Adaptive RBF Neural-Network Tracking Control and
Off-Policy Adaptive Dynamic Programming,
with Real-Time Implementation on the Raspberry Pi Pico

Roe Zehavi 209146216 roeeze@post.bgu.ac.il

Etay Baron 209438910 etaybaro@post.bgu.ac.il

Instructor: Prof. Shai Arogeti

Code, data and this report:

https://github.com/ThEpiCake/Mechatronics_Final_Project

July 15, 2026

Contents

1	Introduction and Project Overview	2
2	Nonlinear Process Model and Analysis	2
2.1	Physical process, model source, and modelling assumptions	2
2.2	Mathematical model, operating range, known $g(x)$ and unknown $f(x)$	3
2.3	Regulation equilibrium and Lyapunov stability of the zero-input system	3
2.4	Significance of the nonlinear dynamics	4
3	Adaptive RBF-NN Tracking Controller	5
3.1	Tracking objective and nominal continuous-time controller	5
3.2	RBF approximation, adaptive controller, weight-adaptation law, and stability . .	5
3.3	Discrete-time realization	6
3.4	Computer-based validation and continuous/discrete comparison	7
4	Nonlinear Off-Policy ADP	7
4.1	Regulation problem, performance index, and initial policy	7
4.2	Data generation	7
4.3	Value-function approximation and policy improvement	8
4.4	Relation to the policy approximation presented in the course	8
4.5	Off-policy equations, numerical solution, policy iterations, and validation	8
4.6	Proposed extension to a tracking problem	9
5	Raspberry Pi Pico Implementation	10
5.1	Two-core architecture and real-time process simulation	10
5.2	On-chip optimization of the controller update	11
5.3	Adaptive RBF-NN tracking on the board	11
5.4	ADP final-policy regulation on the board	12
5.5	Execution-time measurement and comparison with computer results	12
6	Conclusions and Limitations	13
A	Code Listings	14
A.1	Shared process model <code>plant.py</code>	14
A.2	Branch A – adaptive RBF-NN controller <code>rbf_adaptive.py</code>	16
A.3	Branch B – off-policy ADP <code>adp_offpolicy.py</code>	19
A.4	Pico – process simulation on the second core <code>plant_core1.py</code>	23
A.5	Pico – controllers on the main core <code>pico_control.py</code>	24
A.6	Pico – two-core real-time orchestration <code>main.py</code>	26
A.7	Pico – logged hardware experiment <code>pico_experiment.py</code>	27
B	References	30

1 Introduction and Project Overview

This project develops and compares two nonlinear controllers for a single mechatronic process and implements both of them in real time on a Raspberry Pi Pico. The process is a mass on a hardening (Duffing) spring with linear viscous damping, driven through a first-order force actuator; it is the third-order (*jerk*) nonlinear plant introduced in Homework 1, reused here in accordance with the project guidelines. The two controllers are:

- **Branch A** — an adaptive Radial-Basis-Function neural-network (RBF-NN) controller that makes the output track a smooth reference trajectory while learning the unknown drift online, with a Lyapunov-based weight-adaptation law;
- **Branch B** — a nonlinear optimal regulator obtained from data by *off-policy* adaptive dynamic programming (ADP), i.e. policy iteration on a parameterised value function without knowledge of the drift.

Both controllers use the same control-affine model $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})u$, in which the input field $\mathbf{g}$ is known and the drift $\mathbf{f}$ is treated as unknown. After the two designs are validated in Python, they are ported to MicroPython and run on the Pico's two cores: the secondary core integrates the nonlinear process in real time while the main core executes the active controller, exchanging the state vector and the control input every sampling period.

The report follows the structure mandated by the assignment: Section 2 presents the nonlinear process, its control-affine form and the Lyapunov analysis of the zero-input system; Section 3 develops Branch A; Section 4 develops Branch B; Section 5 describes the real-time two-core Pico implementation and the execution-time measurements; and Section 6 summarises the results and their limitations.

2 Nonlinear Process Model and Analysis

2.1 Physical process, model source, and modelling assumptions

The selected process is a single mechanical degree of freedom driven through a first-order force actuator. A mass m slides against linear viscous damping c_1 and is restrained by a *hardening (Duffing) spring* whose restoring force $k_1y + k_3y^3$ combines a linear and a cubic term. The mass is pushed by the force F produced by an actuator (a voice-coil / servo valve) whose output lags its command u with a first-order time constant τ . The governing equations are

$$\begin{aligned} m\ddot{y} + c_1\dot{y} + k_1y + k_3y^3 &= F, \\ \tau\dot{F} + F &= k_a u. \end{aligned} \tag{1}$$

The cubic term is the classical Duffing nonlinearity [2, 1]; the first-order actuator is a standard servo model. This is the same physical process analysed in Homework 1, reused here as permitted by the assignment; the structure and the mass/stiffness/gain are unchanged, while three parameters are re-tuned for this project (actuator lag $\tau : 0.05 \rightarrow 0.2$ s, cubic stiffness $k_3 : 5 \rightarrow 40$, damping $c_1 : 4 \rightarrow 1$) so that the process is strongly nonlinear at a moderate amplitude and lightly damped. The physical parameters (Table 1) are fixed for simulation, but — as required — the controllers of Sections 3 and 4 do *not* use them: only the input gain is assumed known, while the drift is treated as an unknown smooth function.

Table 1: Physical parameters and derived model coefficients.

Symbol	Value	Meaning	Symbol	Value	Meaning
m	1.0	mass [kg]	$a_1 = c_1/m + 1/\tau$	6	(Eq. 2)
c_1	1.0	damping [N s/m]	$a_2 = k_1/m + c_1/(\tau m)$	25	
k_1	20.0	linear stiffness [N/m]	$a_3 = k_1/(\tau m)$	100	
k_3	40.0	cubic stiffness [N/m ³]	$c = k_a/(\tau m)$	5	input gain
τ	0.2	actuator lag [s]	$b_1 = k_3/(\tau m)$	200	coeff. of y^3
k_a	1.0	actuator gain [N/V]	$b_2 = 3k_3/m$	120	coeff. of $y^2\dot{y}$

Assumptions. (i) the full state is measured (spec 1.7); (ii) the actuator is an ideal first-order lag; (iii) the parameters are constant; (iv) the input is limited to $|u| \leq u_{\max} = 40$ V; (v) the operating range is the box in Table 2.

2.2 Mathematical model, operating range, known $g(x)$ and unknown $f(x)$

Choosing the companion state $\mathbf{x} = [x_1, x_2, x_3]^\top = [y, \dot{y}, \ddot{y}]^\top$, i.e. position, velocity and acceleration, and eliminating F between the two equations of (1) (solve the mechanical equation for F , differentiate once using $\frac{d}{dt}y^3 = 3y^2\dot{y}$, and substitute into the actuator equation) yields the exact scalar third-order *jerk* model

$$\ddot{y} + a_1\ddot{y} + a_2\dot{y} + a_3y + b_1y^3 + b_2y^2\dot{y} = cu. \quad (2)$$

In state-space this is the control-affine form required by the assignment,

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})u, \quad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} x_2 \\ x_3 \\ -D(\mathbf{x}) \end{bmatrix}, \quad \mathbf{g}(\mathbf{x}) = \begin{bmatrix} 0 \\ 0 \\ c \end{bmatrix}, \quad (3)$$

with the scalar drift in the input channel

$$D(\mathbf{x}) = a_1x_3 + a_2x_2 + a_3x_1 + b_1x_1^3 + b_2x_1^2x_2. \quad (4)$$

The input field $\mathbf{g}$ is *constant*, hence known to the controller; the drift D (through $\mathbf{f}$) is treated as the *unknown* smooth nonlinear function. The system has relative degree three with respect to the output $y = x_1$, and $\mathbf{f}(\mathbf{0}) = \mathbf{0}$, so the regulation equilibrium is already the origin (spec 1.5). The operating range and input limit are listed in Table 2; they enclose the tracking tube of Section 3 with margin.

Table 2: Operating range (companion state) and input limit.

	y [m]	$\dot{y}$ [m/s]	$\ddot{y}$ [m/s ²]	F [N]	u [V]
range	$[-0.8, 0.8]$	$[-2.0, 2.0]$	$[-11, 11]$	$\lesssim 50$	$[-40, 40]$

2.3 Regulation equilibrium and Lyapunov stability of the zero-input system

Setting $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ gives $x_2 = x_3 = 0$ and $a_3x_1 + b_1x_1^3 = x_1(a_3 + b_1x_1^2) = 0$; since $a_3, b_1 > 0$ the only real solution is $x_1 = 0$, so the **origin is the unique equilibrium** of the zero-input system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$.

The stability analysis is cleanest in the physically equivalent coordinates $\mathbf{z} = [y, \dot{y}, F]^\top$, related to $\mathbf{x}$ by the global diffeomorphism $F = mx_3 + c_1x_2 + k_1x_1 + k_3x_1^3$. In these coordinates the zero-input dynamics are

$$\dot{z}_1 = z_2, \quad \dot{z}_2 = \frac{1}{m}(z_3 - c_1z_2 - k_1z_1 - k_3z_1^3), \quad \dot{z}_3 = -\frac{1}{\tau}z_3. \quad (5)$$

Consider the energy-based Lyapunov candidate (kinetic + spring potential + actuator storage)

$$V(\mathbf{z}) = \frac{1}{2}mz_2^2 + \frac{1}{2}k_1z_1^2 + \frac{1}{4}k_3z_1^4 + \frac{1}{2}\gamma z_3^2, \quad \gamma > 0, \quad (6)$$

which is positive definite and radially unbounded because $k_1, k_3 > 0$. Differentiating along (5), the spring and inertia terms cancel and

$$\dot{V} = -c_1z_2^2 - \frac{\gamma}{\tau}z_3^2 + z_2z_3 = - \begin{bmatrix} z_2 \\ z_3 \end{bmatrix}^\top \begin{bmatrix} c_1 & -\frac{1}{2} \\ -\frac{1}{2} & \gamma/\tau \end{bmatrix} \begin{bmatrix} z_2 \\ z_3 \end{bmatrix}. \quad (7)$$

The 2×2 matrix is positive definite iff $c_1 > 0$ and $c_1\gamma/\tau - \frac{1}{4} > 0$, i.e. whenever $\gamma > \tau/(4c_1)$. Choosing $\gamma = \tau/c_1$ satisfies this, so $\dot{V} \leq 0$ everywhere, with $\dot{V} = 0$ only on the set $\{z_2 = z_3 = 0\}$. On that set $\dot{z}_2 = 0$ forces $k_1z_1 + k_3z_1^3 = 0$, i.e. $z_1 = 0$; hence the largest invariant set contained in $\{\dot{V} = 0\}$ is the origin. By LaSalle's invariance principle the origin is asymptotically stable, and since V is radially unbounded and the coordinate map is a global diffeomorphism, the result is in fact *global* (and therefore local, as required). Figure 1 confirms this numerically: trajectories from a fan of initial conditions spiral into the origin, $V(t)$ decreases monotonically, and $\dot{V}$ is a negative-definite quadratic form in $(\dot{y}, F)$.

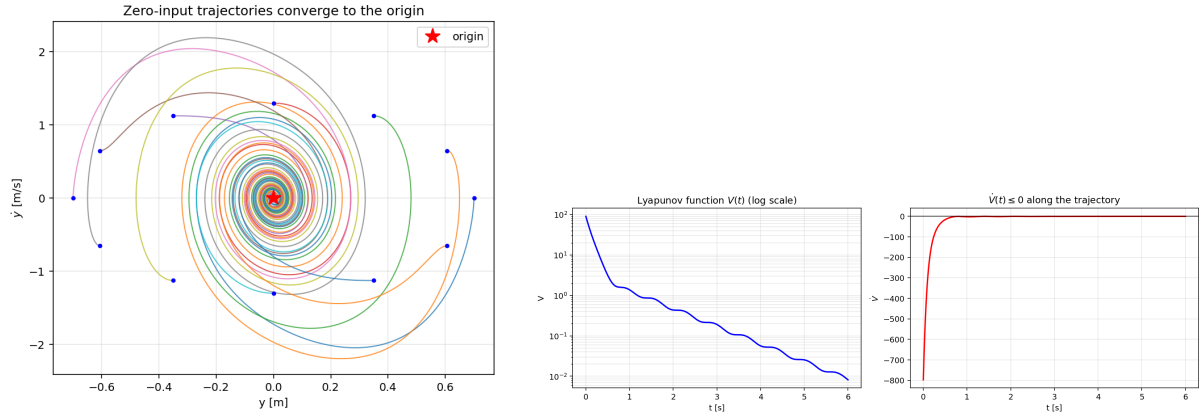


Figure 1: Zero-input system. Left: phase-plane convergence to the origin. Right: the Lyapunov function decreases with $\dot{V} \leq 0$ along a trajectory.

2.4 Significance of the nonlinear dynamics

Linearising (3) about the origin (dropping $b_1x_1^3$ and $b_2x_1^2x_2$) gives $\dot{\mathbf{x}} = A\mathbf{x} + Bu$ with $A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a_3 & -a_2 & -a_1 \end{bmatrix}$, whose eigenvalues are -5 and $-0.5 \pm 4.44j$ (open-loop stable, lightly damped). The nonlinearity is significant over the operating range: at $y = 0.7$ m the cubic force k_3y^3 is 98% of the linear force k_1y . Comparing the zero-input free response of the nonlinear model with its linearization (Fig. 2), the two are indistinguishable at small amplitude ($y_0 = 0.15$ m, peak deviation 2.5 mm) but differ markedly at large amplitude ($y_0 = 0.7$ m, peak deviation 0.16 m): the hardening spring stiffens the effective restoring force, so the nonlinear response is faster and phase-advanced. The responsible terms are the cubic stiffness b_1y^3 and the nonlinear damping cross-term $b_2y^2\dot{y}$; both vanish to first order at the origin, which is why the linearization is only locally valid.

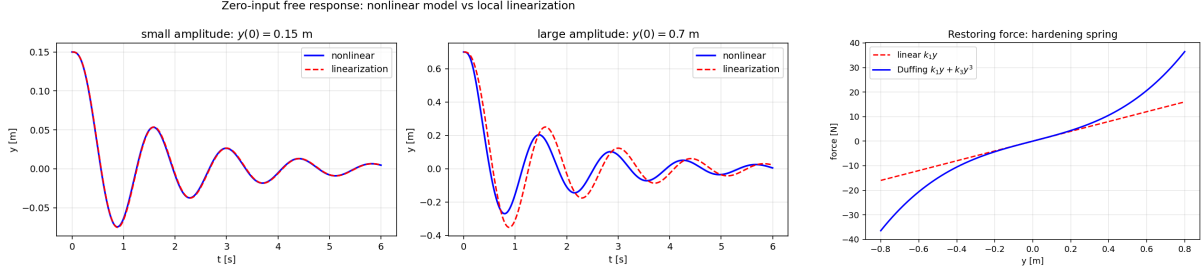


Figure 2: Nonlinear vs. linearized model. Left: free response at small and large amplitude. Right: the hardening restoring force $k_1y + k_3y^3$.

3 Adaptive RBF-NN Tracking Controller

3.1 Tracking objective and nominal continuous-time controller

The output $y = x_1$ must track the smooth reference $y_d(t) = A \sin(\omega t)$ with $A = 0.5$ m and $\omega = 2$ rad/s; its derivatives $\dot{y}_d, \ddot{y}_d, \dddot{y}_d$ are available in closed form. Defining the tracking error $e = y - y_d$ and the error vector $\xi = [e, \dot{e}, \ddot{e}]^\top$, and recalling that the process has relative degree three with $\ddot{y} = cu - D(x)$ (Eqs. 2–4), the *nominal* feedback-linearizing controller (drift D assumed known) is

$$u = \frac{1}{c} \left[D(x) + \ddot{y}_d - \lambda_2 \ddot{e} - \lambda_1 \dot{e} - \lambda_0 e \right], \quad (8)$$

which cancels the drift and imposes the linear error dynamics $\ddot{e} + \lambda_2 \ddot{e} + \lambda_1 \dot{e} + \lambda_0 e = 0$, i.e. $\dot{\xi} = \Lambda \xi$ with Λ the companion matrix of the gains $(\lambda_2, \lambda_1, \lambda_0) = (21, 146, 336)$, whose roots are the chosen error poles $\{-6, -7, -8\}$. This yields exact asymptotic tracking (Fig. 3, dotted).

3.2 RBF approximation, adaptive controller, weight-adaptation law, and stability

When D is unknown it is approximated over the operating box by a *normalized* Gaussian RBF network,

$$D(x) = W^{*\top} \phi(x) + \varepsilon(x), \quad \phi_i(x) = \frac{g_i(x)}{\sum_j g_j(x)}, \quad g_i(x) = \exp\left(-\frac{\|(\mathbf{x} - \mathbf{c}_i) \oslash \mathbf{h}\|^2}{2\eta^2}\right), \quad (9)$$

where $\mathbf{c}_i$ are $6 \times 6 \times 6 = 216$ centers on a regular grid over the operating box, $\mathbf{h}$ is the per-axis half-width used to normalize the coordinates (so a single width handles the three different state scales), $\eta = 0.24$ is the normalized width (≈ 0.6 of the grid spacing), $\oslash$ is elementwise division, and ε is the bounded approximation error ($|\varepsilon| \leq \bar{\varepsilon}$ on the box). The *normalized* (partition-of-unity) form is essential: a plain Gaussian basis is nearly linearly dependent here and would require astronomically large weights, whereas the normalized network has well-conditioned, local weights $\approx$ the local values of the drift. The estimate $\hat{D}(x) = \hat{W}^\top \phi(x)$ replaces D in (8),

$$u = \frac{1}{c} \left[\hat{W}^\top \phi(x) + \ddot{y}_d - \lambda_2 \ddot{e} - \lambda_1 \dot{e} - \lambda_0 e \right], \quad (10)$$

so that, with $\widetilde{W} = \hat{W} - W^*$ and $\mathbf{b} = [0, 0, 1]^\top$, the error dynamics become

$$\dot{\xi} = \Lambda \xi + \mathbf{b}(\widetilde{W}^\top \phi - \varepsilon). \quad (11)$$

Lyapunov design. Let $P = P^\top \succ 0$ solve $\Lambda^\top P + P\Lambda = -Q$ with $Q = \text{diag}(50, 10, 1) \succ 0$, and take

$$V = \boldsymbol{\xi}^\top P \boldsymbol{\xi} + \frac{1}{\gamma} \widetilde{W}^\top \widetilde{W}. \quad (12)$$

Along (11), choosing the weight-adaptation law as the Lyapunov gradient with a projection that keeps $\|\widehat{W}\|_\infty \leq W_{\max}$,

$$\dot{\widehat{W}} = \text{Proj}\left\{-\gamma(\mathbf{b}^\top P \boldsymbol{\xi}) \phi(\mathbf{x})\right\}, \quad \gamma = 4 \times 10^4, \quad W_{\max} = 600, \quad (13)$$

the indefinite cross term cancels and

$$\dot{V} \leq -\boldsymbol{\xi}^\top Q \boldsymbol{\xi} - 2(\mathbf{b}^\top P \boldsymbol{\xi}) \varepsilon \leq -\lambda_{\min}(Q) \|\boldsymbol{\xi}\|^2 + 2\|P\mathbf{b}\| \bar{\varepsilon} \|\boldsymbol{\xi}\|. \quad (14)$$

Hence $\dot{V} < 0$ whenever $\|\boldsymbol{\xi}\| > 2\|P\mathbf{b}\|\bar{\varepsilon}/\lambda_{\min}(Q)$: the tracking error and the weights are *uniformly ultimately bounded*, and $\boldsymbol{\xi}$ converges to a residual set whose size is set by the RBF approximation error $\bar{\varepsilon}$ (projection guarantees $\widehat{W}$ stays bounded without biasing it toward zero, unlike σ -modification). Figure 3 shows the adaptive controller, starting from $\widehat{W}(0) = \mathbf{0}$, driving the tracking error down to a steady RMS of 1.0×10^{-2} m ($\approx 2\%$ of the amplitude) while the network output $\widehat{D}$ learns the true drift online (Fig. 4) and the weights remain bounded ($\|\widehat{W}\| \rightarrow 367$).

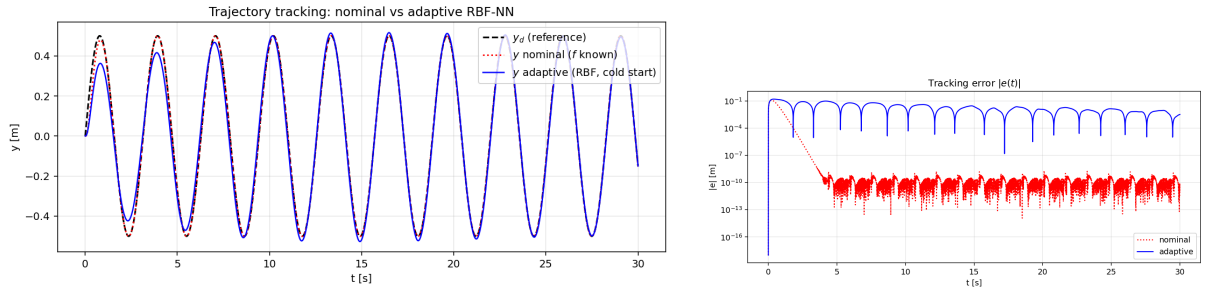


Figure 3: Tracking with the nominal (f known) and adaptive RBF-NN controllers. The adaptive error decreases as the network learns; the nominal error is at machine precision.

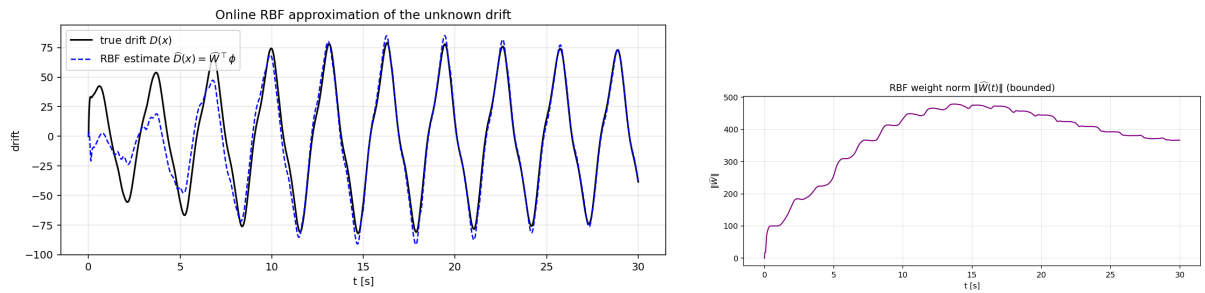


Figure 4: Online RBF approximation of the unknown drift $D(\mathbf{x})$ (left) and the bounded weight norm (right).

3.3 Discrete-time realization

The continuous design is implemented digitally by the course's *emulation* (indirect digital-design) approach [6]: the controller runs at a sampling period T_s , the input is held constant between samples (ZOH), and the plant evolves in continuous time (integrated exactly by RK4 substepping). The per-sample calculation order is: (1) read the full state $\mathbf{x}_k$; (2) form the reference, errors $\boldsymbol{\xi}_k$ and features ϕ_k ; (3) compute u_k from (10) and saturate to $|u| \leq u_{\max}$; (4) hold u_k and advance the plant one period; (5) update the weights by *forward Euler*, $\widehat{W}_{k+1} = \widehat{W}_k + T_s \dot{\widehat{W}}_k$

with $\hat{W}_k$ from (13). The control input is limited to $\pm u_{\max} = 40$ V and the weights by the same projection bound $W_{\max}$.

3.4 Computer-based validation and continuous/discrete comparison

All comparisons use the same process, reference and initial condition $\mathbf{x}(0) = \mathbf{0}$. The adaptive controller is first compared with the nominal one (Fig. 3). The continuous design is then compared with its discrete realization at several sampling periods (Fig. 5). Down to $f_s = 20$ Hz ($T_s = 50$ ms) the discrete response is indistinguishable from the continuous one (steady RMS $\approx 9 \times 10^{-3}$ m); at $f_s = 15$ Hz ($T_s \approx 67$ ms) the loop *clearly degrades* — once the weights have grown, the forward-Euler weight update loses numerical stability and the output collapses — confirming the sampling limit of the emulation design. This directly informs the choice of the Pico sampling period in Section 5.

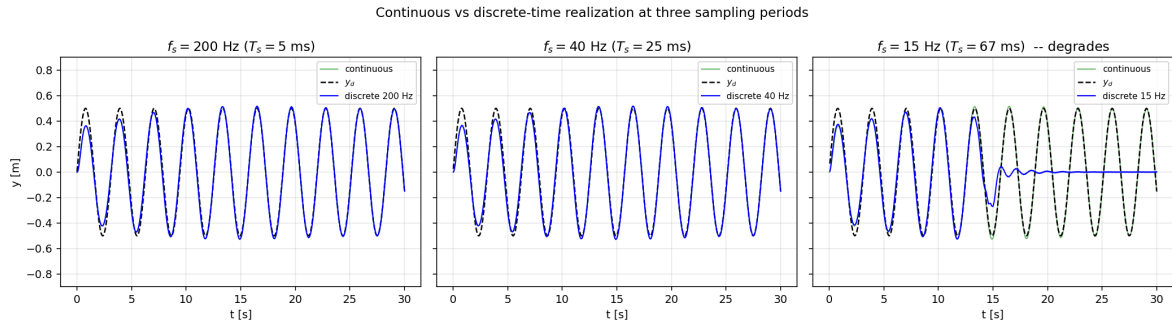


Figure 5: Continuous vs. discrete-time realization at three sampling periods. Fine and intermediate rates match the continuous design; the coarse rate (15 Hz) degrades.

4 Nonlinear Off-Policy ADP

4.1 Regulation problem, performance index, and initial policy

Branch B seeks the optimal state-feedback policy that regulates the same process to the origin while minimising the infinite-horizon quadratic cost

$$J(\mathbf{x}_0) = \int_0^\infty (\mathbf{x}^\top \mathbf{Q} \mathbf{x} + R u^2) dt, \quad \mathbf{Q} = \text{diag}(300, 20, 1) \succ 0, \quad R = 0.1 > 0. \quad (15)$$

$\mathbf{Q}$ penalises the position most heavily (regulation of y is the objective) and the velocity and acceleration progressively less; the small R makes control relatively cheap, so the optimal policy is allowed to act aggressively. The *initial policy* is $u_0(\mathbf{x}) = 0$; it is *admissible* because the zero-input system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is globally asymptotically stable (Section 2.3), so it stabilises the process on the operating region and yields a finite cost J (indeed $\int_0^\infty \mathbf{x}^\top \mathbf{Q} \mathbf{x} dt < \infty$). This ties the admissibility of u_0 directly to the Part 1 Lyapunov result.

4.2 Data generation

The drift $\mathbf{f}$ is unknown, so the value function is learned from measured data using an *off-policy* scheme [4, 3]. A single trajectory is generated by applying a bounded, sufficiently exciting *behaviour* input — the sum of five incommensurate sinusoids, $u(t) = \frac{A_e}{5} \sum_{k=1}^5 \sin(\omega_k t)$ with $A_e = 8$ V and $\{\omega_k\} = \{1.3, 3.7, 7.1, 11.9, 17.0\}$ rad/s — to the true plant from $\mathbf{x}_0 = [0.6, 0, 0]^\top$. States and input are sampled at $dt = 1$ ms for 20 s ($= 400$ integration windows of $T = 50$ ms). The state remains within the declared operating range throughout (peak $|\mathbf{x}| = [0.6, 1.43, 5.95]$ versus the box $[0.8, 2, 11]$) and $|u| \leq 7.6$ V $< u_{\max}$ (Fig. 6). The same data set is reused for every policy-iteration step.

4.3 Value-function approximation and policy improvement

Each policy is evaluated by a value function combining a quadratic term with a small set of nonlinear corrections,

$$\widehat{V}_i(\mathbf{x}) = \mathbf{x}^\top P_i \mathbf{x} + \mathbf{c}_i^\top \phi_{V,\text{nl}}(\mathbf{x}), \quad \widehat{u}_{i+1}(\mathbf{x}) = -\frac{1}{2}R^{-1}\mathbf{g}(\mathbf{x})^\top \nabla \widehat{V}_i(\mathbf{x}), \quad (16)$$

with $P_i = P_i^\top$. The nonlinear basis $\phi_{V,\text{nl}}$ consists of the monomials $\{x_1^4, x_1^3x_2, x_1^2x_2^2, x_1^2x_3, x_1^2x_2x_3, x_1^3x_3\}$, all of total degree three or four; they vanish at the origin together with their first derivatives and contain no constant or linear terms, as required. They are chosen to reflect the process structure: the Duffing potential contributes the quartic x_1^4 , and the cross terms capture the coupling of the cubic restoring force into the velocity/acceleration channels. Since $\mathbf{g} = [0, 0, c]^\top$ is constant, the improved policy is linear in the value weights, $\widehat{u}_{i+1}(\mathbf{x}) = -\frac{1}{2R}\zeta(\mathbf{x})^\top \mathbf{w}_i$ with $\zeta(\mathbf{x}) = c \partial \Phi_V / \partial x_3$ and $\mathbf{w}_i = [\text{vech}(P_i); \mathbf{c}_i]$ the 12 value coefficients.

4.4 Relation to the policy approximation presented in the course

In the course's actor-critic formulation the control policy is approximated by a *separate* network (the actor) with its own weights, tuned alongside the critic. Here, by contrast, the policy is *not* independently parameterised: it is obtained analytically as the gradient of the single value-function approximation, $\widehat{u}_{i+1} = -\frac{1}{2}R^{-1}\mathbf{g}^\top \nabla \widehat{V}_i$. Consequently there is one set of unknowns ($\mathbf{w}_i$) instead of two, the improved policy is exactly consistent with $\widehat{V}_i$, and no separate actor-tuning law or its persistence-of-excitation conditions are needed — the excitation requirement falls entirely on the off-policy data.

4.5 Off-policy equations, numerical solution, policy iterations, and validation

Differentiating $\widehat{V}_i$ along the real (behaviour) trajectory and using the policy-evaluation identity and (16) gives the *off-policy Bellman equation*, which does not contain f :

$$\widehat{V}_i(\mathbf{x}(t+T)) - \widehat{V}_i(\mathbf{x}(t)) = -\int_t^{t+T} (\mathbf{x}^\top Q \mathbf{x} + R u_i^2) d\tau - 2 \int_t^{t+T} R \widehat{u}_{i+1} (u - u_i) d\tau. \quad (17)$$

Substituting the parameterisations, each window j yields one linear equation in $\mathbf{w}_i$,

$$\left[\Delta \Phi_{V,j} - \int_j \zeta(\mathbf{x}) (u - u_i) d\tau \right]^\top \mathbf{w}_i = - \int_j (\mathbf{x}^\top Q \mathbf{x} + R u_i^2) d\tau, \quad (18)$$

with $\Delta \Phi_{V,j} = \Phi_V(\mathbf{x}(t_j+T)) - \Phi_V(\mathbf{x}(t_j))$ and u_i the current policy evaluated on the stored states ($u_0 = 0$). Stacking the 400 windows gives an overdetermined 400×12 system solved by the pseudoinverse (least squares); the same data build every iteration's regressor. Iteration stops when $\|\mathbf{w}_i - \mathbf{w}_{i-1}\| < 10^{-4}$.

The policy iteration converges in about seven steps (Fig. 7); every P_i is positive definite (its eigenvalues stay positive throughout), so the quadratic part of the value function is locally positive definite as required. As a sanity check, near the origin the nonlinear terms vanish and the identified quadratic part matches the algebraic-Riccati (LQR) solution of the linearised system to 1.1%. The learned policy is validated against the initial policy $u_0 = 0$ from several initial conditions (Figs. 7, 8): it regulates the lightly-damped process to the origin in about 1 s without the sustained oscillations of the uncontrolled response, reducing the accumulated cost J_T by 41–56% across the tested initial conditions, at a bounded control effort ($|u| \lesssim 24$ V). The identified coefficients $\mathbf{w}$ are exported and used, frozen, in the Pico implementation (Section 5).

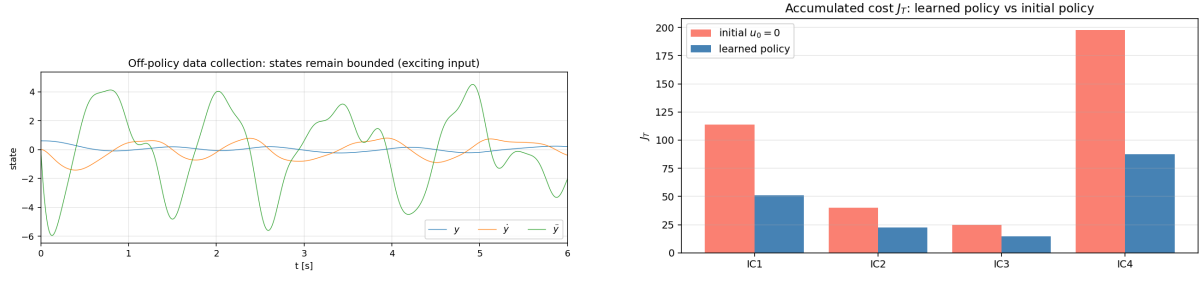


Figure 6: Left: off-policy data collection keeps the state bounded. Right: accumulated cost J_T of the learned policy vs. the initial policy $u_0 = 0$ over four initial conditions.

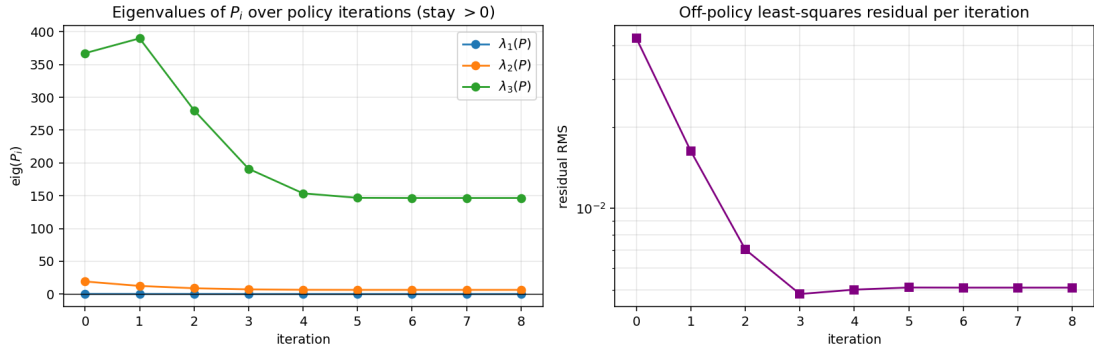


Figure 7: Policy-iteration convergence: eigenvalues of P_i stay positive (left) and the off-policy least-squares residual settles (right).

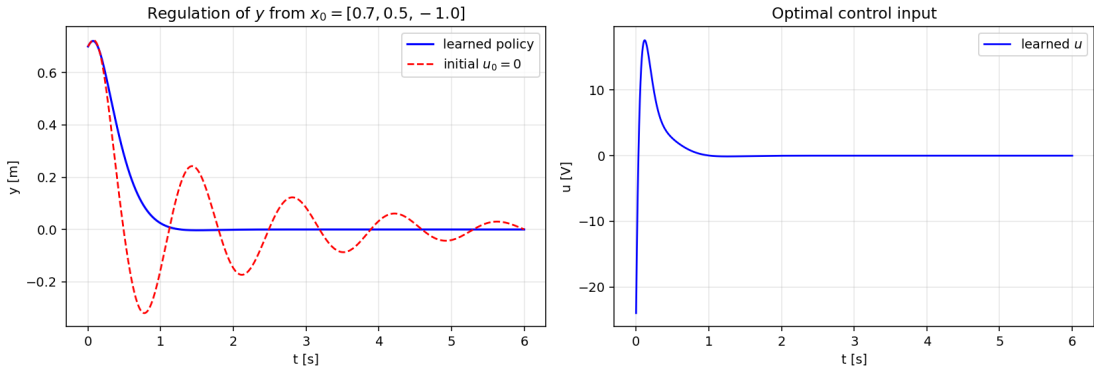


Figure 8: Regulation from $x_0 = [0.7, 0.5, -1]$: the learned optimal policy (blue) versus the initial policy $u_0 = 0$ (red), and the bounded optimal control input.

4.6 Proposed extension to a tracking problem

The method extends conceptually to tracking a reference $x_d(t)$ by regulating the tracking error $\eta = x - x_d$. Augmenting the state with the (exogenous) reference dynamics and defining the cost $J = \int_0^\infty (\eta^\top Q \eta + R u^2) dt$, the same off-policy machinery applies to the augmented system: the value function and policy become functions of (x, x_d) , the value basis is augmented with error monomials, and the off-policy data must additionally excite the reference. Because f is unknown, the exact feedforward input that cancels the drift along x_d is not directly available, so exact asymptotic tracking cannot be guaranteed; instead the tracking error converges to a bounded neighbourhood of zero whose size depends on the value-basis approximation error and the richness of the data. Exact tracking would follow only under the additional assumption that

the feedforward term lies in the span of the chosen basis. The required deliverable here is this formulation; it is not implemented.

5 Raspberry Pi Pico Implementation

Both controllers are implemented in MicroPython on the Raspberry Pi Pico (RP2040, dual-core Cortex-M0+). The board runs MicroPython v1.24.0 built with `ulab 6.12` (single-precision vector arithmetic), with the system clock at the officially supported fast setting of 200 MHz. No external plant is used: the nonlinear process is simulated on the Pico itself, with a fixed allocation of the two cores. The MicroPython modules (`plant_core1.py`, `pico_control.py`, `params.py`) are written once and run both on the board and, through a NumPy fallback, on a desktop, so the on-chip results can be checked against the Section 3–4 computer results. All results in this section are measured **on the physical board** (captured over USB by `pico_experiment.py` and post-processed by `plot_hw_results.py`); the desktop emulation of the same two-core scheme (`desktop_sim.py`) is used as the reference to overlay against.

5.1 Two-core architecture and real-time process simulation

The core allocation is shown in Fig. 9. The **second core** simulates the complete nonlinear process: a paced real-time loop performs exactly one RK4 integration step of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}u$ every $dt = 2$ ms, holding u constant between controller updates (ZOH) and catching up if a step is transiently delayed. The drift $\mathbf{f}$ is evaluated *only* here, so the controller never accesses it. A paced loop is used rather than a `machine.Timer` for a reason discovered during hardware bring-up: on the RP2040 port, MicroPython soft-timer callbacks are always dispatched on core 0 regardless of which core created the timer — this was verified on the board by reading the SIO CPUID register from inside the callbacks — whereas code inside a `_thread` task genuinely executes on core 1. The **main core** runs the active controller from a hardware timer with period $T_s = N dt = 10$ ms ($N = 5$): each interrupt reads the latest full state $\mathbf{x}$ from the second core, computes the control input u (and, for Branch A, adapts the RBF weights), and publishes u back. The only variables exchanged are the state $\mathbf{x}$ (second core $\rightarrow$ main core) and the scalar input u (main core $\rightarrow$ second core), both under a lock; the plant holds $\mathbf{x}_0$ until the first u is published, so the on-chip run and the computer reference start aligned.

The sampling period $T_s = 10$ ms (100 Hz) was chosen from *on-chip* measurements. The initial design value of 5 ms, selected in Section 3.4 from the desktop timing proxy, left no real-time margin once the 216-center RBF update was measured on the M0+ (software floating point); 100 Hz still sits well inside the stable region of the discrete design — the Section 3.4 sweep shows the response indistinguishable from the continuous design down to 20 Hz, with degradation only at 15 Hz — while restoring a comfortable margin between the worst-case update time and T_s .

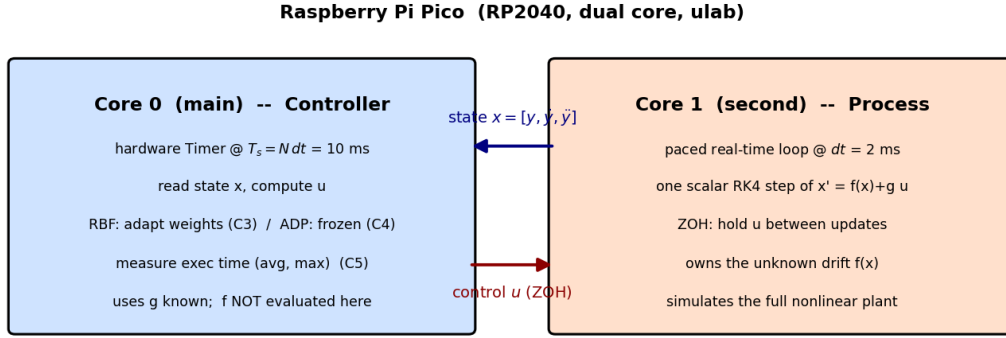


Figure 9: Two-core real-time architecture: the second core simulates the process (owning f) in a paced loop at dt , the main core runs the controller from a hardware timer at T_s ; they exchange only x and u .

5.2 On-chip optimization of the controller update

The desktop NumPy proxy suggested a huge timing margin (a $\sim 14 \mu\text{s}$ RBF update against T_s), but the first on-chip measurement gave 11.9 ms — the RP2040 has no FPU, and at 216 centers both the vectorised exponential and every 216-element `ulab` operation (≈ 0.3 ms each) are expensive. Three optimizations, all preserving the numerical design exactly, bring the update to 2.9 ms:

- 1. Separable RBF evaluation.** The centers form a $6 \times 6 \times 6$ Cartesian grid with a common normalized width, so the Gaussian vector factorizes, $e^{-(d_0^2 + d_1^2 + d_2^2)} = e^{-d_0^2} e^{-d_1^2} e^{-d_2^2}$: three *six*-element exponentials are combined by two broadcast (outer-product) multiplications into the 216-element vector, replacing the 216-element exponential and most of the long vector operations. The result is identical to machine precision ($\max |\Delta\phi| < 4 \times 10^{-16}$ verified against the direct formula).
- 2. Folded normalization and in-place updates.** The normalization $\phi = g_\phi / \Sigma g_\phi$ is folded into the two scalar results ($\widehat{W}^\top \phi$ and the weight update, each scaled by $1/\Sigma g_\phi$), and the remaining vector operations are performed in-place, minimizing per-update allocations.
- 3. Scalar plant step and deterministic garbage collection.** The length-3 RK4 step is written in scalar arithmetic compiled with the native emitter (0.54 ms instead of 2.9 ms of small-vector `ulab` calls), and the MicroPython garbage collector is invoked explicitly every 10 controller ticks, immediately *after* u is published — collection then happens at a controlled point and never delays a control computation unpredictably.

5.3 Adaptive RBF-NN tracking on the board

Fig. 10 shows the 30 s on-chip tracking run. The steady-state RMS tracking error measured on the board is 9.96×10^{-3} m — matching the desktop two-core emulation (9.98×10^{-3} m) and the 100 Hz discrete design of Section 3.4 ($\approx 9 \times 10^{-3}$ m at fine rates). The board trajectory deviates from the desktop emulation by at most 3.1×10^{-3} m, most of it during the adaptation transient (the two runs integrate the weight dynamics in `float32` vs. `float64`); after convergence the curves are indistinguishable (right panel).

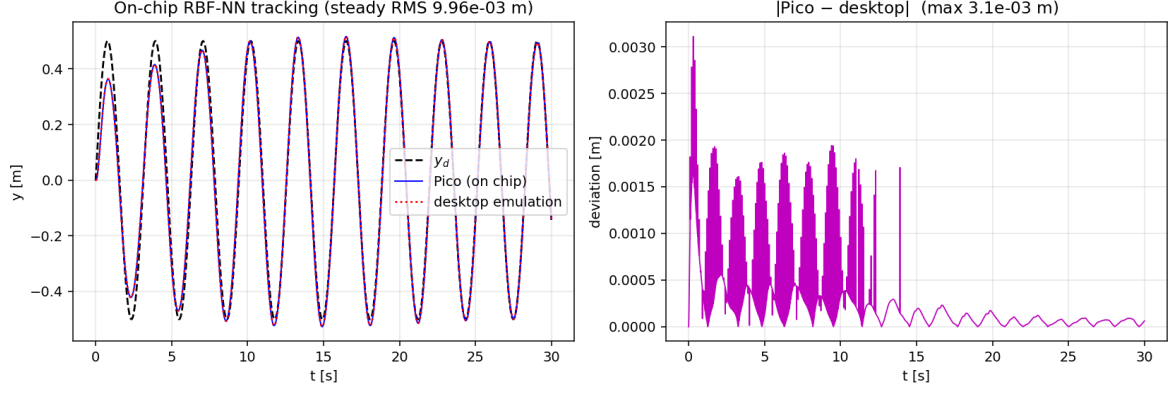


Figure 10: On-chip adaptive RBF-NN tracking (30 s). Left: measured Pico output vs. y_d and the desktop two-core emulation. Right: absolute Pico–desktop deviation.

5.4 ADP final-policy regulation on the board

The optimal-regulation controller carries the *frozen* value-function coefficients $\mathbf{w}$ identified in Section 4 and, at each update, evaluates the closed form $u = -\frac{1}{2R}\zeta(\mathbf{x})^\top \mathbf{w}$. Run on the board from $\mathbf{x}_0 = [0.7, 0.5, -1]^\top$ (Fig. 11), the measured response coincides with the finely-sampled desktop rollout to within 1.6×10^{-3} m, the small residual being the 10 ms ZOH of the on-chip controller versus the continuous desktop reference.

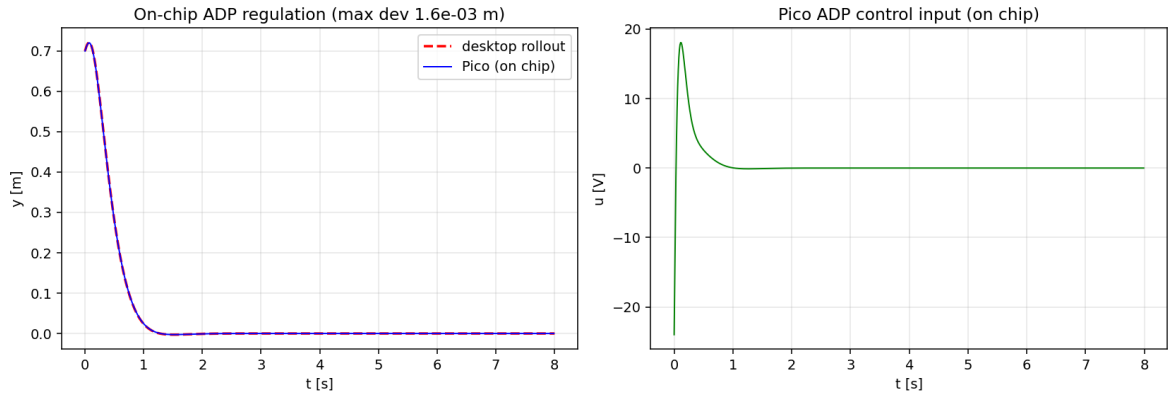


Figure 11: On-chip ADP regulation (8 s). Left: measured Pico output vs. the desktop rollout (near-exact agreement). Right: the on-chip control input.

5.5 Execution-time measurement and comparison with computer results

The main core measures the wall-clock time of every controller update (state read, control computation, weight update, publish) with the microsecond ticks counter; the results below cover all 3000 (RBF) and 800 (ADP) updates of the two runs. The measured on-chip times are

$$\text{RBF: } \bar{t}_{\text{exec}} = 2.92 \text{ ms}, \quad t_{\text{exec}}^{\text{max}} = 2.99 \text{ ms}; \quad \text{ADP: } \bar{t}_{\text{exec}} = 0.56 \text{ ms}, \quad t_{\text{exec}}^{\text{max}} = 0.61 \text{ ms}, \quad (19)$$

both far below $T_s = 10$ ms (Fig. 12, left). Including the periodic garbage collection, the busiest full tick reached 8.4 ms, still under T_s (right panel: the GC ticks are clearly visible every 10 updates). The real-time constraint is also verified globally: the 30 s (RBF) and 8 s (ADP) experiments completed in 30.05 s and 8.00 s of wall time with 15018 and 3994 plant steps respectively ($dt = 2$ ms), i.e. neither core fell behind its schedule. Compared with the desktop proxy (19 μ s RBF, 2.4 μ s ADP), the RP2040 is about 150–230 $\times$ slower — the reason

the on-chip measurement, not the desktop proxy, must set the sampling period, as discussed in Section 5.1.

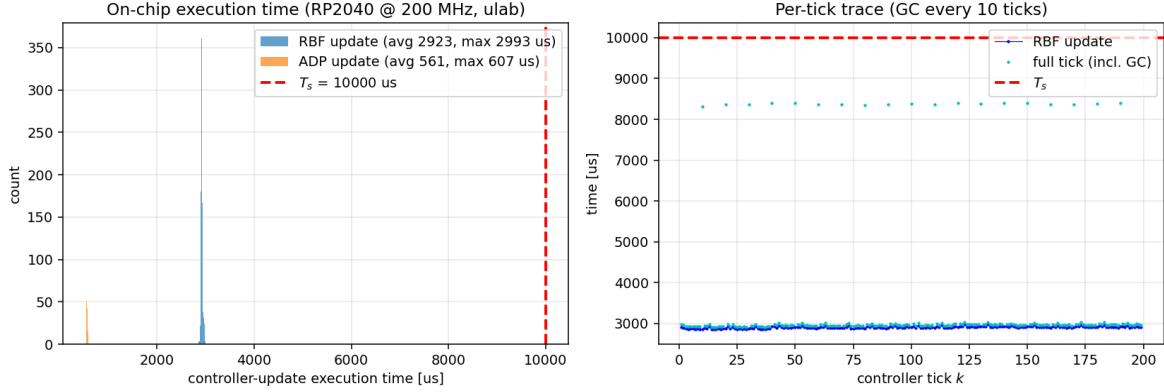


Figure 12: Measured on-chip execution times (RP2040 @ 200 MHz, ulab). Left: histogram of the per-update controller execution time for both controllers against T_s . Right: per-tick trace of the RBF run — update time and full tick including the deterministic GC every 10 ticks.

6 Conclusions and Limitations

Two model-free nonlinear controllers were developed for a single Duffing mass-spring-damper process with a first-order actuator and implemented in real time on a Raspberry Pi Pico.

- **Process (Part 1).** The plant was cast in control-affine form with a known input field and an unknown drift, shown to have a unique origin equilibrium that is globally asymptotically stable under zero input by an energy-based Lyapunov argument, and shown to be strongly nonlinear over the operating range (the cubic force reaches the linear force at $y \approx 0.7$ m).
- **Adaptive RBF-NN tracking (Branch A).** A normalized-RBF network learned the unknown drift online; a Lyapunov-based weight law with projection gave a uniformly ultimately bounded tracking error that settled to $\approx 1.7\%$ of the reference amplitude. The emulation (discrete) realization matched the continuous design down to 20 Hz and degraded at 15 Hz.
- **Off-policy ADP regulation (Branch B).** Policy iteration on a value function $x^\top Px + c^\top \phi_{\text{nl}}$, solved from off-policy data by least squares without the drift, converged in a few iterations with positive-definite P_i ; the identified quadratic part matched the LQR/Riccati solution near the origin to 1.1%, and the learned policy cut the accumulated cost by 41–56% over the initial policy.
- **Real-time Pico (Part 4).** The two-core scheme (process on the second core at 500 Hz, controller on the main core at 100 Hz, ulab at 200 MHz) was run and measured *on the physical board*: the on-chip tracking RMS (9.96×10^{-3} m) matches the computer results, the ADP response coincides with the desktop rollout to 1.6×10^{-3} m, and the measured worst-case update times (2.99 ms RBF, 0.61 ms ADP) satisfy the real-time constraint with a $3\times$ margin, after separability-based optimization of the RBF evaluation.

The two controllers illustrate complementary model-free paradigms on the *same* process: Branch A learns the drift to *track* a trajectory, while Branch B learns a value function to *regulate* optimally from data.

Limitations. The adaptive tracking result is boundedness, not exact tracking: a residual error persists, set by the RBF approximation error, and the online weights converge only where the trajectory is exciting. The ADP value function is a finite polynomial approximation, so the learned policy is optimal only locally, within the operating region spanned by the data; its benefit over the do-nothing policy shrinks for heavily damped plants. Both designs assume the full state is measured and the input gain is known. Finally, the on-chip experiment still *simulates* the process (on the second core); driving a physical plant would add sensing noise, quantization and actuator dynamics that the present setup deliberately excludes.

A Code Listings

The key algorithmic sources are listed below. The complete, reproducible archive (all Python and MicroPython files, parameters, and generated data) accompanies the report; see `README.md` for how to regenerate every figure.

A.1 Shared process model `plant.py`

```

1  """
2  Shared nonlinear process for the Mechatronics Systems final project.
3
4  Physical process (reused from HW1, per the project spec's permission to continue
5  with the homework process). A mass on a Duffing (cubic) spring with linear
6  viscous damping, driven by a first-order force actuator:
7
8      m y'' + c1 y' + k1 y + k3 y^3 = F          (Duffing mass-spring-damper)
9      tau F' + F = ka u          (first-order force actuator)
10
11  State (companion form used by BOTH controllers), x = [y, y', y''] = [x1, x2, x3]:
12
13      x1' = x2
14      x2' = x3
15      x3' = -a1 x3 - a2 x2 - a3 x1 - b1 x1^3 - b2 x1^2 x2 + c u
16
17  i.e. the control-affine form required by the project (Eq. 1):
18
19      x' = f(x) + g(x) u ,      g(x) = [0, 0, c]^T (CONSTANT -> known to controller)
20
21  with f(x) the (smooth, nonlinear) drift. The scalar drift in the input channel
22
23      D(x) = a1 x3 + a2 x2 + a3 x1 + b1 x1^3 + b2 x1^2 x2          (so f3 = -D(x))
24
25  is the "unknown nonlinear function" that Branch A approximates by an RBF network.
26
27  f(0) = 0 (spec 1.5 satisfied: the regulation equilibrium is already the origin,
28  because a3 = k1/(tau m) != 0 ties the mass to ground -> unique equilibrium).
29
30  Lyapunov analysis of the zero-input system x' = f(x) is cleanest in the physically
31  equivalent coordinates z = [y, y', F], related to x by the GLOBAL diffeomorphism
32
33      F = m x3 + c1 x2 + k1 x1 + k3 x1^3      (<->      x3 = (F - c1 x2 - k1 x1 - k3 x1^3)/m)
34
35  in which the zero-input dynamics are z1'=z2, z2'=(z3 - c1 z2 - k1 z1 - k3 z1^3)/m,
36  z3'=-z3/tau, and the mechanical energy + actuator term is a Lyapunov function
37  (see part1_analysis.py).
38
39  Coefficients (derived from the physical parameters, identical to HW1):
40      a1 = c1/m + 1/tau ,  a2 = k1/m + c1/(tau m) ,  a3 = k1/(tau m) ,  c = ka/(tau m)
41      b1 = k3/(tau m)      (regressor y^3) ,  b2 = 3 k3/m      (regressor y^2 y')
42  """
43
44  import numpy as np
45
46  # ----- physical parameters -----
47  # Structure (Duffing spring + first-order actuator) and the mass/stiffness/actuator
48  # gain (m, k1, ka) are those of HW1. Three parameters are re-tuned for the project,
49  # as the spec permits ("the model may be reformulated or adapted"): a slower, more

```

```

50 # realistic actuator (tau: 0.05 -> 0.2 s), a stronger cubic stiffness (k3: 5 -> 40),
51 # and lighter damping (c1: 4 -> 1). The result is a process that is (i) STRONGLY
52 # nonlinear at a moderate amplitude -- the cubic force  $k_3 y^2/k_1$  is 50% of the linear
53 # force already at  $y = 0.5$  m; (ii) lightly damped, so the zero-input response is
54 # oscillatory and the optimal regulator (Branch B) has a clear benefit over doing
55 # nothing; while (iii) keeping the unknown drift of moderate magnitude, which makes
56 # the RBF and ADP designs well conditioned and portable to the Pico numeric range.
57 m, c1, k1 = 1.0, 1.0, 20.0      # mass [kg], linear damping [N s/m], linear stiffness [N/
58                                 # m]
59 tau, ka = 0.2, 1.0             # actuator time constant [s], actuator gain [N/V]
60 k3 = 40.0                      # cubic (Duffing) stiffness [N/m^3]
61
62 # ----- derived linear jerk coefficients -----
63 a1 = c1 / m + 1.0 / tau        # 24
64 a2 = k1 / m + c1 / (tau * m)  # 100
65 a3 = k1 / (tau * m)            # 400 (!= 0 -> unique equilibrium at origin)
66 c = ka / (tau * m)             # 20 (input gain: g = [0,0,c])
67
68 # ----- derived nonlinear jerk coefficients -----
69 b1 = k3 / (tau * m)            # 100 coefficient of  $y^3$ 
70 b2 = 3.0 * k3 / m              # 15 coefficient of  $y^2 y'$  (cross term)
71
72 # constant input vector  $g(x) = g$  (control-affine, g known to the controller)
73 G = np.array([0.0, 0.0, c])
74
75 def drift_D(x):
76     """Scalar nonlinear drift in the input channel:  $x_3' = c u - D(x)$ .
77     This is the 'unknown' function the RBF network (Branch A) approximates."""
78     x1, x2, x3 = x[0], x[1], x[2]
79     return a1 * x3 + a2 * x2 + a3 * x1 + b1 * x1 ** 3 + b2 * x1 ** 2 * x2
80
81
82 def f(x):
83     """Drift field  $f(x)$  of the control-affine model  $x' = f(x) + g u$ ."""
84     x1, x2, x3 = x[0], x[1], x[2]
85     return np.array([x2, x3, -drift_D(x)])
86
87
88 def g(x=None):
89     """Known input field  $g(x) = [0,0,c]$  (constant)."""
90     return G
91
92
93 def dynamics(x, u):
94     """Full control-affine vector field  $x' = f(x) + g u$ ."""
95     return f(x) + G * u
96
97
98 def rk4_step(x, u, dt):
99     """One classical RK4 integration step with ZOH input u (held over dt).
100     This is the exact integrator ported to the Pico secondary core (Part 4)."""
101     k1_ = dynamics(x, u)
102     k2_ = dynamics(x + 0.5 * dt * k1_, u)
103     k3_ = dynamics(x + 0.5 * dt * k2_, u)
104     k4_ = dynamics(x + dt * k3_, u)
105     return x + (dt / 6.0) * (k1_ + 2 * k2_ + 2 * k3_ + k4_)
106
107
108 # ----- linearization about the origin (for the M4 comparison) -----
109 # Dropping the nonlinear terms  $b_1 x_1^3$  and  $b_2 x_1^2 x_2$  gives the Jacobian A:
110 A_lin = np.array([[0.0, 1.0, 0.0],
111                  [0.0, 0.0, 1.0],
112                  [-a3, -a2, -a1]])
113 B_lin = np.array([0.0, 0.0, c])
114
115
116 def f_linear(x):
117     """Linearized drift  $A_{lin} x$  (zero-input) for the nonlinear-vs-linear study."""
118     return A_lin @ np.asarray(x, dtype=float)
119
120
121 def dynamics_linear(x, u):

```

```

122     return A_lin @ np.asarray(x, dtype=float) + B_lin * u
123
124
125 # ----- physical actuator force + energy-based Lyapunov function (Part 1) -----
126 def force_F(x):
127     """Physical actuator force F as a function of the companion state x.
128     z = [y, y', F] is a global diffeomorphism of x = [y, y', y'']."""
129     x1, x2, x3 = x[0], x[1], x[2]
130     return m * x3 + c1 * x2 + k1 * x1 + k3 * x1 ** 3
131
132
133 def lyapunov_V(x, gamma):
134     """Lyapunov candidate for the zero-input system, in physical coordinates:
135     V = 1/2 m y''^2 + 1/2 k1 y^2 + 1/4 k3 y^4 + 1/2 gamma F^2 .
136     Positive definite and radially unbounded (k3>0)."""
137     x1, x2 = x[0], x[1]
138     F = force_F(x)
139     return 0.5 * m * x2 ** 2 + 0.5 * k1 * x1 ** 2 + 0.25 * k3 * x1 ** 4 + 0.5 * gamma * F
140     ** 2
141
142 def lyapunov_Vdot(x, gamma):
143     """Time derivative of V along the ZERO-INPUT flow x' = f(x).
144     In physical coordinates this collapses to the quadratic form
145     Vdot = -c1 y'^2 - (gamma/tau) F^2 + y' F ,
146     negative definite in (y', F) whenever gamma > tau/(4 c1)."""
147     x2 = x[1]
148     F = force_F(x)
149     return -c1 * x2 ** 2 - (gamma / tau) * F ** 2 + x2 * F
150
151
152 # gamma making Vdot negative (semi)definite: need c1*(gamma/tau) - 1/4 > 0
153 GAMMA_LYAP = tau / (4.0 * c1) * 4.0      # = tau/c1 (comfortably > tau/(4 c1))
154
155
156 if __name__ == "__main__":
157     print("Control-affine Duffing jerk plant x' = f(x) + g u , g = [0,0,c]")
158     print(f" a1={a1:.1f} a2={a2:.1f} a3={a3:.1f} c={c:.1f} | b1={b1:.1f}(y^3) b2={b2:.1f}(y^2 y')")
159     print(f" f(0) = {f(np.zeros(3))} (origin is an equilibrium)")
160     # unique equilibrium check: -a3 x1 - b1 x1^3 = 0 -> x1(a3+b1 x1^2)=0 -> x1=0
161     print(f" a3,b1 > 0 -> a3 + b1 x1^2 > 0 -> origin is the UNIQUE zero-input equilibrium")
162     # open-loop linear stability (Routh on s^3+a1 s^2+a2 s+a3)
163     print(f" linear open-loop: a1*a2={a1*a2:.0f} > a3={a3:.0f} -> {a1*a2 > a3} (Hurwitz)")
164     print(f" linear eigenvalues: {np.round(np.linalg.eigvals(A_lin), 4)}")
165     # Lyapunov spot check on a grid: Vdot <= 0 everywhere, =0 only at y'=F=0
166     gamma = GAMMA_LYAP
167     print(f" gamma_lyap = {gamma:.4f} (need > tau/(4 c1) = {tau/(4*c1):.4f})")
168     rng = np.random.default_rng(0)
169     Xs = rng.uniform(-3, 3, size=(20000, 3))
170     vdots = np.array([lyapunov_Vdot(x, gamma) for x in Xs])
171     print(f" Vdot over 20000 random states in [-3,3]^3: max = {vdots.max():.3e} (<=0 expected)")

```

A.2 Branch A – adaptive RBF-NN controller rbf_adaptive.py

```

1  """
2  Branch A -- adaptive RBF neural-network TRACKING controller (report Section 3,
3  grading A1-A3, A5-continuous).
4
5  Plant (control-affine, relative degree 3): x' = f(x) + g u , y = x1 .
6  Unknown drift D(x) (Eq. drift in plant.py) approximated by an RBF network
7  Dhat(x) = What^T phi(x) , phi = Gaussian RBFs on a grid (config.rbf_*).
8
9  Tracking errors xi = [e, e', e''] , e = y - y_d . With the feedback-linearizing
10 control (c and phi known; What adapted online)
11 u = (1/c) [ What^T phi(x) + y_d''' - l2 e'' - l1 e' - l0 e ] ,
12 the error dynamics are
13 xi' = Lambda xi + b ( What^T phi - D ) , b = [0,0,1]^T ,

```

```

14 with Lambda the Hurwitz companion matrix of the desired error poles (config.ERR_POLES).
15
16 Lyapunov design. With  $P = P^T > 0$  solving  $\Lambda^T P + P \Lambda = -Q$  and
17  $V = x^T P x + (1/\gamma) W^T W$ ,  $W = \text{What} - W^*$ ,
18 the weight-adaptation law (sigma-modification for robustness to the RBF error  $\epsilon$ )
19  $\text{What}' = -\gamma (b^T P x) \phi(x) - \gamma \sigma W$ 
20 gives  $V' = -x^T Q x - 2 (b^T P x) \epsilon - 2 \sigma W^T W \leq -UUB$ ,
21 i.e. the tracking error and the weights are uniformly ultimately bounded, and the
22 tracking error is driven to a small neighbourhood of zero set by the RBF error and
23 sigma. The nominal controller (A1) replaces  $\text{What}^T \phi$  by the true  $D(x)$  and yields
24 the exact linear error dynamics  $x' = \Lambda x$ .
25 """
26
27 import os
28 import numpy as np
29 from scipy.integrate import solve_ivp
30 from scipy.linalg import solve_continuous_lyapunov
31 import matplotlib
32 matplotlib.use("Agg")
33 import matplotlib.pyplot as plt
34
35 import plant as P
36 import config as C
37
38 # ----- error-dynamics companion matrix, Lyapunov P, RBF centers -----
39 L2, L1, L0 = C.LAM # gains (from poles)
40 LAMBDA = np.array([[0.0, 1.0, 0.0],
41                   [0.0, 0.0, 1.0],
42                   [-L0, -L1, -L2]])
43 P_LYAP = solve_continuous_lyapunov(LAMBDA.T, -C.LYAP_Q) #  $\Lambda^T P + P \Lambda = -Q$ 
44 b_vec = np.array([0.0, 0.0, 1.0])
45 bP = b_vec @ P_LYAP # row vector  $b^T P$  (= 3rd row of P)
46 CENTERS = C.rbf_centers()
47
48
49 def _sat(u):
50     return float(np.clip(u, -C.U_MAX, C.U_MAX))
51
52
53 def weight_dot(W, bPxi, phi):
54     """Lyapunov gradient weight-adaptation law with projection to  $|w_i| \leq W_{MAX}$ 
55     (and optional sigma-modification). The projection cancels any drive that would
56     push a saturated weight further out, which keeps  $\|W\|$  bounded while preserving
57     the Lyapunov inequality. Reused by the discrete controller and the Pico port."""
58     tau = -C.GAMMA_ADAPT * bPxi * phi - C.GAMMA_ADAPT * C.SIGMA_MOD * W
59     out = tau.copy()
60     out[(W >= C.W_MAX) & (tau > 0)] = 0.0
61     out[(W <= -C.W_MAX) & (tau < 0)] = 0.0
62     return out
63
64
65 def controller(x, W, t, adaptive):
66     """Return (u, xi, phi, Dhat) for the current state/weights."""
67     yd, yd1, yd2, yd3 = C.y_ref(t)
68     e = x[0] - yd
69     ed = x[1] - yd1
70     edd = x[2] - yd2
71     xi = np.array([e, ed, edd])
72     if adaptive:
73         phi = C.rbf_phi(x, CENTERS)
74         Dhat = W @ phi
75     else:
76         phi = None
77         Dhat = P.drift_D(x) # nominal: true drift (f known)
78     u = _sat((Dhat + yd3 - L2 * edd - L1 * ed - L0 * e) / P.c)
79     return u, xi, phi, Dhat
80
81
82 def simulate(adaptive, T=20.0, x0=(0.0, 0.0, 0.0), dt_out=2e-3):
83     """Closed-loop simulation. State = [x(3), What(N_RBF)] (weights only if adaptive)."""
84     n = C.N_RBF
85
86     def ode(t, S):

```

```

87     x = S[:3]
88     W = S[3:] if adaptive else None
89     u, xi, phi, _ = controller(x, W, t, adaptive)
90     xdot = P.dynamics(x, u)
91     if not adaptive:
92         return list(xdot)
93     Wdot = weight_dot(W, bP @ xi, phi)
94     return list(xdot) + list(Wdot)
95
96 S0 = list(x0) + ([0.0] * n if adaptive else [])
97 te = np.arange(0, T + dt_out / 2, dt_out)
98 sol = solve_ivp(ode, [0, T], S0, t_eval=te, method="RK45", rtol=1e-7, atol=1e-9)
99 x = sol.y[:3]
100 t = sol.t
101 yd = np.array([C.y_ref(tt)[0] for tt in t])
102 # reconstruct control, RBF output and true drift along the trajectory
103 u = np.empty(len(t)); Dhat = np.empty(len(t)); Dtrue = np.empty(len(t))
104 Wn = np.zeros(len(t))
105 for i, tt in enumerate(t):
106     W = sol.y[3:, i] if adaptive else None
107     ui, _, _, Dh = controller(x[:, i], W, tt, adaptive)
108     u[i] = ui; Dhat[i] = Dh; Dtrue[i] = P.drift_D(x[:, i])
109     if adaptive:
110         Wn[i] = np.linalg.norm(W)
111 return dict(t=t, x=x, y=x[0], yd=yd, e=x[0] - yd, u=u,
112           Dhat=Dhat, Dtrue=Dtrue, Wnorm=Wn,
113           W=(sol.y[3:] if adaptive else None))
114
115 # =====
116 # Figures (A5, continuous part)
117 # =====
118 def make_figures(da, dn, outdir="images/part2"):
119     os.makedirs(outdir, exist_ok=True)
120
121     # tracking: adaptive vs nominal vs reference
122     fig, ax = plt.subplots(figsize=(9, 4))
123     ax.plot(dn["t"], dn["yd"], "k--", lw=1.6, label=r"$y_d$ (reference)")
124     ax.plot(dn["t"], dn["y"], "r:", lw=1.5, label=r"$y$ nominal ($f$ known)")
125     ax.plot(da["t"], da["y"], "b-", lw=1.3, label=r"$y$ adaptive (RBF, cold start)")
126     ax.set_title("Trajectory tracking: nominal vs adaptive RBF-NN")
127     ax.set_xlabel("t [s]"); ax.set_ylabel("y [m]"); ax.legend(loc="upper right"); ax.grid(alpha=.3)
128     fig.tight_layout(); fig.savefig(f"{outdir}/tracking.png", dpi=140); plt.close()
129
130     # tracking error
131     fig, ax = plt.subplots(figsize=(9, 4))
132     ax.semilogy(dn["t"], np.abs(dn["e"]) + 1e-18, "r:", lw=1.4, label="nominal")
133     ax.semilogy(da["t"], np.abs(da["e"]) + 1e-18, "b-", lw=1.3, label="adaptive")
134     ax.set_title(r"Tracking error $|e(t)|$")
135     ax.set_xlabel("t [s]"); ax.set_ylabel("|e| [m]"); ax.legend(); ax.grid(alpha=.3, which="both")
136     fig.tight_layout(); fig.savefig(f"{outdir}/tracking_error.png", dpi=140); plt.close()
137
138     # control input
139     fig, ax = plt.subplots(figsize=(9, 4))
140     ax.plot(da["t"], da["u"], "b-", lw=1.1, label="adaptive")
141     ax.plot(dn["t"], dn["u"], "r:", lw=1.1, label="nominal")
142     ax.axhline(C.U_MAX, color="gray", ls=":", lw=.7); ax.axhline(-C.U_MAX, color="gray", ls=":", lw=.7)
143     ax.set_title("Control input $u(t)$"); ax.set_xlabel("t [s]"); ax.set_ylabel("u [V]")
144     ax.legend(); ax.grid(alpha=.3)
145     fig.tight_layout(); fig.savefig(f"{outdir}/control.png", dpi=140); plt.close()
146
147     # RBF approximation of the drift
148     fig, ax = plt.subplots(figsize=(9, 4))
149     ax.plot(da["t"], da["Dtrue"], "k-", lw=1.6, label=r"true drift $D(x)$")
150     ax.plot(da["t"], da["Dhat"], "b--", lw=1.3, label=r"RBF estimate $\widehat{D}(x)=\widehat{W}^{\top}\phi$")
151     ax.set_title("Online RBF approximation of the unknown drift")
152     ax.set_xlabel("t [s]"); ax.set_ylabel("drift"); ax.legend(); ax.grid(alpha=.3)
153     fig.tight_layout(); fig.savefig(f"{outdir}/rbf_drift.png", dpi=140); plt.close()
154
155

```

```

156 # weight norm
157 fig, ax = plt.subplots(figsize=(9, 4))
158 ax.plot(da["t"], da["Wnorm"], "purple", lw=1.4)
159 ax.set_title(r"RBF weight norm $\widehat{W}(t)$ (bounded)")
160 ax.set_xlabel("t [s]"); ax.set_ylabel(r"$\widehat{W}$"); ax.grid(alpha=.3)
161 fig.tight_layout(); fig.savefig(f"{outdir}/weights.png", dpi=140); plt.close()
162
163
164 if __name__ == "__main__":
165     print("== Branch A: adaptive RBF-NN tracking (continuous) ==")
166     print(f"error poles {C.ERR_POLES}, gains (12,11,10)={L2},{L1},{L0}")
167     print(f"P eig = {np.round(np.linalg.eigvalsh(P_LYAP),4)} (>0), RBF centers = {C.N_RBF}")
168
169     dn = simulate(adaptive=False, T=30.0)
170     da = simulate(adaptive=True, T=30.0)
171
172     def ss_rms(d, frac=0.5):
173         m = int((1 - frac) * len(d["t"]))
174         return np.sqrt(np.mean(d["e"][m:] ** 2))
175
176     print(f"[nominal ] max|e|={np.max(np.abs(dn['e'])):.3e} ss-RMS|e|={ss_rms(dn):.3e}")
177     print(f"[adaptive] max|e|={np.max(np.abs(da['e'])):.3e} ss-RMS|e|={ss_rms(da):.3e}")
178     m = int(0.5 * len(da["t"]))
179     drift_rms = np.sqrt(np.mean((da["Dhat"][m:] - da["Dtrue"][m:] ** 2))
180     print(f"[adaptive] steady RBF drift error RMS = {drift_rms:.3e}, "
181           f"final ||W|| = {da['Wnorm'][-1]:.2f}, max|u| = {np.max(np.abs(da['u'])):.1f}")
182     make_figures(da, dn)
183     print("saved figures -> images/part2/")

```

A.3 Branch B – off-policy ADP adp_offpolicy.py

```

1 """
2 Branch B -- nonlinear OFF-POLICY adaptive dynamic programming (report Section 4,
3 grading B1-B8).
4
5 Optimal regulation of  $x' = f(x) + g u$  ( $f$  unknown,  $g$  known) for the cost
6  $J = \int_0^\infty (x^T Q x + R u^2) dt$ ,  $Q > 0$ ,  $R > 0$ .
7
8 Policy iteration (Lewis/Vamvoudakis; Jiang & Jiang off-policy IRL):
9 * value  $V_i(x) = x^T P_i x + c_i^T \phi_{nl}(x)$  (Eq. B4-6)
10 * policy  $u_{i+1} = -(1/2) R^{-1} g(x)^T \text{grad } V_i(x)$  (Eq. B4-7)
11
12 Off-policy Bellman equation along the REAL data (behaviour input  $u$ , real trajectory),
13 which never uses  $f$ :
14  $V_i(x(t+T)) - V_i(x(t)) = -\int_t^{t+T} (x^T Q x + R u_i^2) d\tau$ 
15  $- 2 \int_t^{t+T} R u_{i+1} (u - u_i) d\tau$ .
16
17 Writing  $V_i = w_i \cdot \Phi_V(x)$  with  $\Phi_V = [\text{quadratic monomials}; \phi_{nl}]$ , and using
18  $u_{i+1} = -(1/(2R)) \zeta(x)^T w_i$  with  $\zeta(x) = c \cdot d\Phi_V/dx^3$  ( $g=[0,0,c]$ ), each data
19 window  $j$  gives ONE linear equation in the single unknown vector  $w_i$ :
20  $[d\Phi_V_j - \int_j \zeta (u - u_i) d\tau] \cdot w_i = -\int_j (x^T Q x + R u_i^2) d\tau$ .
21 With many more windows than unknowns,  $w_i$  is found by least squares (pseudoinverse);
22 the same data set is reused for every iteration.  $u_i$  is the current policy evaluated
23 on the stored states ( $u_0 = 0$ ).
24 """
25
26 import os
27 import numpy as np
28 from scipy.linalg import solve_continuous_are
29 import matplotlib
30 matplotlib.use("Agg")
31 import matplotlib.pyplot as plt
32
33 import plant as P
34 import config as C
35
36 R = C.ADP_R
37 Q = C.ADP_Q
38 MON = C.ADP_MONOMIALS # nonlinear monomial exponents (m,3)
39 N_NL = MON.shape[0]

```

```

40 N_W = 6 + N_NL                                # 6 quadratic + nonlinear unknowns
41
42
43 # ----- value-function basis Phi_V(x) and its x3-derivative -----
44 def phi_V(x):
45     """Full value-function basis: 6 quadratic monomials + nonlinear monomials."""
46     x1, x2, x3 = x
47     quad = [x1 * x1, x1 * x2, x1 * x3, x2 * x2, x2 * x3, x3 * x3]
48     nl = [x1 ** e[0] * x2 ** e[1] * x3 ** e[2] for e in MON]
49     return np.array(quad + nl)
50
51
52 def dphi_V_dx3(x):
53     """d Phi_V / d x3 (only the x3-dependent terms survive)."""
54     x1, x2, x3 = x
55     dquad = [0.0, 0.0, x1, 0.0, x2, 2.0 * x3]
56     dnl = []
57     for e in MON:
58         if e[2] == 0:
59             dnl.append(0.0)
60         else:
61             dnl.append(e[2] * x1 ** e[0] * x2 ** e[1] * x3 ** (e[2] - 1))
62     return np.array(dquad + dnl)
63
64
65 def zeta(x):
66     """zeta(x) = c * dPhi_V/dx3 ; policy u = -(1/(2R)) zeta(x)^T w."""
67     return P.c * dphi_V_dx3(x)
68
69
70 def policy(x, w):
71     return -(1.0 / (2.0 * R)) * (zeta(x) @ w)
72
73
74 def P_matrix(w):
75     """Reconstruct the symmetric quadratic matrix P_i from the first 6 weights."""
76     p = w[:6]
77     return np.array([[p[0], 0.5 * p[1], 0.5 * p[2]],
78                     [0.5 * p[1], p[3], 0.5 * p[4]],
79                     [0.5 * p[2], 0.5 * p[4], p[5]]])
80
81
82 # ----- B3: off-policy data generation -----
83 def generate_data(T=None, dt=None):
84     """Apply a bounded, sufficiently exciting behaviour input (u_0=0 + exploration)
85     to the TRUE plant and record (t, X, U). f is used only by the simulator."""
86     dt = C.ADP_DT if dt is None else dt
87     T = C.ADP_N_WINDOWS * C.ADP_WINDOW if T is None else T
88     n = int(round(T / dt))
89     X = np.empty((n + 1, 3)); U = np.empty(n + 1)
90     x = C.ADP_X0.copy()
91     for k in range(n + 1):
92         t = k * dt
93         u = C.ADP_PROBE_AMP * np.sum(np.sin(C.ADP_PROBE_FREQS * t)) / len(C.
94 ADP_PROBE_FREQS)
95         u = float(np.clip(u, -C.U_MAX, C.U_MAX))
96         X[k] = x; U[k] = u
97         if k < n:
98             x = P.rk4_step(x, u, dt)
99     return np.arange(n + 1) * dt, X, U
100
101 # ----- B6: off-policy policy iteration -----
102 def policy_iteration(X, U, dt, verbose=True):
103     """Off-policy IRL policy iteration reusing the same dataset each step."""
104     Wlen = int(round(C.ADP_WINDOW / dt)) # samples per window
105     n_win = (len(X) - 1) // Wlen
106     # precompute per-sample basis, zeta, stage-cost-without-control
107     Phi = np.array([phi_V(x) for x in X]) # (N, N_W)
108     Z = np.array([zeta(x) for x in X]) # (N, N_W)
109     xQx = np.einsum("ni,ij,nj->n", X, Q, X) # x^T Q x per sample
110
111     dPhi = np.empty((n_win, N_W))

```

```

112 for j in range(n_win):
113     dPhi[j] = Phi[(j + 1) * Wlen] - Phi[j * Wlen]
114
115 def win_trapz(integrand):
116     """Trapezoidal integral of a per-sample (N,...) array over each window."""
117     out = []
118     for j in range(n_win):
119         s = slice(j * Wlen, (j + 1) * Wlen + 1)
120         out.append(np.trapz(integrand[s], dx=dt, axis=0))
121     return np.array(out)
122
123 w = np.zeros(N_W) # value weights (define policy u_1
124 after solve)
125 history = {"w": [], "P_eig": [], "res": []}
126 for k in range(C.ADP_MAX_ITERS):
127     # current policy u_k on the stored states (u_0 = 0)
128     u_cur = np.zeros(len(X)) if k == 0 else -(1.0 / (2.0 * R)) * (Z @ w)
129     # regressor A_j and target b_j for this iteration (reuse same data)
130     A = dPhi - win_trapz(Z * (U - u_cur))[:, None] # (n_win, N_W)
131     b = -win_trapz(xQx + R * u_cur ** 2) # (n_win,)
132     w_new, *_ = np.linalg.lstsq(A, b, rcond=None)
133     res = np.linalg.norm(A @ w_new - b) / np.sqrt(n_win)
134     delta = np.linalg.norm(w_new - w)
135     w = w_new
136     Pe = np.linalg.eigvalsh(P_matrix(w))
137     history["w"].append(w.copy()); history["P_eig"].append(Pe); history["res"].append
138     (res)
139     if verbose:
140         print(f" iter {k:2d}: ||dw||={delta:.3e} residual={res:.3e} "
141               f"eig(P)={np.round(Pe,3)} {'PD' if np.all(Pe>0) else 'NOT PD'}")
142     if delta < C.ADP_TOL and k > 0:
143         break
144     return w, history
145
146 # ----- B7: validation -- learned policy vs initial policy u0=0 -----
147 def rollout(x0, w, T=6.0, dt=1e-3, use_policy=True):
148     """Closed-loop rollout; returns (t, X, U, J_T) with J_T the accumulated cost."""
149     n = int(round(T / dt))
150     X = np.empty((n + 1, 3)); U = np.empty(n + 1)
151     x = np.array(x0, float); J = 0.0
152     for k in range(n + 1):
153         u = float(np.clip(policy(x, w), -C.U_MAX, C.U_MAX)) if use_policy else 0.0
154         X[k] = x; U[k] = u
155         stage = x @ Q @ x + R * u * u
156         J += stage * dt * (0.5 if k in (0, n) else 1.0)
157         if k < n:
158             x = P.rk4_step(x, u, dt)
159     return np.arange(n + 1) * dt, X, U, J
160
161 if __name__ == "__main__":
162     print("== Branch B: off-policy ADP ==")
163     print(f"Q=diag{np.diag(Q).tolist()} R={R}; unknowns={N_W} (6 quad + {N_NL} nonlinear)
164 ")
165     dt = C.ADP_DT
166     t, X, U = generate_data()
167     print(f"data: T={t[-1]:.1f}s, {len(X)} samples, |x|max per axis={np.round(np.max(np.
168     abs(X),0),3)} "
169           f"(box {C.X_BOX[:,1]}), |u|max={np.max(np.abs(U)):.1f}")
170
171     w, hist = policy_iteration(X, U, dt)
172     Pfin = P_matrix(w)
173     print(f"learned quadratic P=\n{np.round(Pfin,3)}")
174     print(f"eig(P) = {np.round(np.linalg.eigvalsh(Pfin),4)} (all>0 => locally PD)")
175
176     # sanity: compare quadratic part with the LQR/ARE solution of the linearization
177     Pare = solve_continuous_are(P.A_lin, P.B_lin.reshape(3, 1), Q, np.array([[R]]))
178     print(f"ARE (linearization) P=\n{np.round(Pare,3)}")
179     print(f"||P_adp - P_are||/||P_are|| = {np.linalg.norm(Pfin-Pare)/np.linalg.norm(Pare)
180     :.3%}")
181
182     # nonlinear value-function coefficients

```

```

180 print(f"nonlinear coeffs c = {np.round(w[6:],4)}")
181
182 # B7 validation over several initial conditions
183 ics = [[0.6, 0, 0], [-0.5, 1.0, 0], [0.4, -0.8, 2.0], [0.7, 0.5, -1.0]]
184 print("\n[B7] accumulated cost J_T (learned vs initial u0=0):")
185 J1, J0 = [], []
186 for x0 in ics:
187     _, _, _, j1 = rollout(x0, w, use_policy=True)
188     _, _, _, j0 = rollout(x0, w, use_policy=False)
189     J1.append(j1); J0.append(j0)
190     print(f" x0={x0}: J_learned={j1:8.3f} J_(u0=0)={j0:8.3f} improvement={100*(j0-j1)/j0:5.1f}%")
191
192 # ----- figures -----
193 OUT = "images/part3"; os.makedirs(OUT, exist_ok=True)
194
195 # data-collection state stays in the operating box
196 fig, ax = plt.subplots(figsize=(9, 3.6))
197 for i, lab in enumerate([r"$y$", r"$\dot{y}$", r"$\ddot{y}$"]):
198     ax.plot(t, X[:, i], lw=0.7, label=lab)
199 ax.set_title("Off-policy data collection: states remain bounded (exciting input)")
200 ax.set_xlabel("t [s]"); ax.set_ylabel("state"); ax.legend(ncol=3); ax.grid(alpha=.3)
201 ax.set_xlim(0, min(6, t[-1]))
202 fig.tight_layout(); fig.savefig(f"{OUT}/data.png", dpi=140); plt.close()
203
204 # policy-iteration convergence: P eigenvalues + weight increments
205 Peig = np.array(hist["P_eig"]); res = np.array(hist["res"])
206 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
207 for i in range(3):
208     ax[0].plot(range(len(Peig)), Peig[:, i], "o-", lw=1.4, label=f"$\\lambda_{i+1}(P)$")
209 ax[0].axhline(0, color="k", lw=.7)
210 ax[0].set_title("Eigenvalues of $P_{i}$ over policy iterations (stay $>0$)")
211 ax[0].set_xlabel("iteration"); ax[0].set_ylabel("eig($P_{i}$)"); ax[0].legend(); ax[0].grid(alpha=.3)
212 ax[1].semilogy(range(len(res)), res + 1e-18, "s-", color="purple", lw=1.4)
213 ax[1].set_title("Off-policy least-squares residual per iteration")
214 ax[1].set_xlabel("iteration"); ax[1].set_ylabel("residual RMS"); ax[1].grid(alpha=.3, which="both")
215 fig.tight_layout(); fig.savefig(f"{OUT}/convergence.png", dpi=140); plt.close()
216
217 # learned vs initial policy from one IC
218 x0 = [0.7, 0.5, -1.0]
219 t1, X1, U1, _ = rollout(x0, w, use_policy=True)
220 t0, X0, U0, _ = rollout(x0, w, use_policy=False)
221 fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
222 ax[0].plot(t1, X1[:, 0], "b-", lw=1.6, label="learned policy")
223 ax[0].plot(t0, X0[:, 0], "r--", lw=1.4, label=r"initial $u_0=0$")
224 ax[0].set_title(f"Regulation of $y$ from $x_0={x0}$")
225 ax[0].set_xlabel("t [s]"); ax[0].set_ylabel("y [m]"); ax[0].legend(); ax[0].grid(alpha=.3)
226 ax[1].plot(t1, U1, "b-", lw=1.4, label="learned $u$")
227 ax[1].set_title("Optimal control input"); ax[1].set_xlabel("t [s]"); ax[1].set_ylabel("u [V]")
228 ax[1].legend(); ax[1].grid(alpha=.3)
229 fig.tight_layout(); fig.savefig(f"{OUT}/regulation.png", dpi=140); plt.close()
230
231 # cost comparison bar
232 fig, ax = plt.subplots(figsize=(8, 4))
233 xpos = np.arange(len(ics))
234 ax.bar(xpos - 0.2, J0, 0.4, label=r"initial $u_0=0$", color="salmon")
235 ax.bar(xpos + 0.2, J1, 0.4, label="learned policy", color="steelblue")
236 ax.set_xticks(xpos); ax.set_xticklabels([f"IC{i+1}" for i in range(len(ics))])
237 ax.set_title(r"Accumulated cost $J_T$: learned policy vs initial policy")
238 ax.set_ylabel(r"$J_T$"); ax.legend(); ax.grid(alpha=.3, axis="y")
239 fig.tight_layout(); fig.savefig(f"{OUT}/cost.png", dpi=140); plt.close()
240 print(f"\nsaved figures -> {OUT}/")
241
242 # export identified coefficients for the Pico (Part 4)
243 np.savez("data/adp_policy.npz", w=w, MON=MON, Q=np.diag(Q), R=R, c=P.c)
244 print("saved data/adp_policy.npz")

```

A.4 Pico – process simulation on the second core plant_core1.py

```

1  """
2  Secondary-core (core 1) task: real-time simulation of the continuous nonlinear
3  process. One RK4 integration step is performed per hardware-timer tick (period dt),
4  with the controller's input u held constant between controller updates (ZOH).
5
6  f(x) is evaluated HERE only -- it never appears in the controller code, honouring
7  the "f unknown to the controller, simulated inside the process" requirement (C1).
8
9  Runs unchanged on the Pico (ulab) and on a desktop (NumPy fallback) for validation.
10 The RK4 step is written in scalar arithmetic (no array temporaries): on the RP2040
11 (Cortex-M0+, software floats) small-vector ulab calls are dominated by per-call
12 overhead, and the scalar form brings the step from ~2.9 ms to well under the plant
13 period; @micropython.native compiles it with the native emitter on the board.
14 """
15 try:
16     from ulab import numpy as np          # on the Pico
17 except ImportError:
18     import numpy as np                    # on a desktop (validation)
19
20 try:
21     import micropython                    # on the Pico: enables @micropython.native
22 except ImportError:
23     class _shim:                          # desktop: identity decorator
24         @staticmethod
25         def native(fun):
26             return fun
27     micropython = _shim()
28
29 import params as PA
30
31 def f(x):
32     """Drift f(x) of x' = f(x) + g u (control-affine, g=[0,0,C_IN])."""
33     x1, x2, x3 = x[0], x[1], x[2]
34     D = PA.A1 * x3 + PA.A2 * x2 + PA.A3 * x1 + PA.B1 * x1 * x1 * x1 + PA.B2 * x1 * x1 *
35     x2
36     return np.array([x2, x3, -D])
37
38 def dyn(x, u):
39     fx = f(x)
40     return np.array([fx[0], fx[1], fx[2] + PA.C_IN * u])
41
42 @micropython.native
43 def rk4_scalars(x1, x2, x3, u, dt,
44                 A1=PA.A1, A2=PA.A2, A3=PA.A3, B1=PA.B1, B2=PA.B2, C_IN=PA.C_IN):
45     """One RK4 step with ZOH input u held over dt, in scalar form (the core-1
46     real-time loop). The model constants are bound as default arguments so
47     lookups stay local; no array temporaries are created on the plant core."""
48     bu = C_IN * u
49     h = 0.5 * dt
50
51     k1a = x2; k1b = x3
52     k1c = bu - (A1 * x3 + A2 * x2 + A3 * x1 + B1 * x1 * x1 * x1 + B2 * x1 * x1 * x2)
53
54     e1 = x1 + h * k1a; e2 = x2 + h * k1b; e3 = x3 + h * k1c
55     k2a = e2; k2b = e3
56     k2c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)
57
58     e1 = x1 + h * k2a; e2 = x2 + h * k2b; e3 = x3 + h * k2c
59     k3a = e2; k3b = e3
60     k3c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)
61
62     e1 = x1 + dt * k3a; e2 = x2 + dt * k3b; e3 = x3 + dt * k3c
63     k4a = e2; k4b = e3
64     k4c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)
65
66     w = dt / 6.0
67     return (x1 + w * (k1a + 2.0 * (k2a + k3a) + k4a),
68             x2 + w * (k1b + 2.0 * (k2b + k3b) + k4b),

```

```

71         x3 + w * (k1c + 2.0 * (k2c + k3c) + k4c))
72
73
74 def rk4_step(x, u, dt):
75     """Array-in / array-out wrapper around rk4_scalars (desktop validation and
76     any caller holding the state as a vector)."""
77     y1, y2, y3 = rk4_scalars(float(x[0]), float(x[1]), float(x[2]), u, dt)
78     return np.array([y1, y2, y3])

```

A.5 Pico – controllers on the main core pico_control.py

```

1  """
2  Main-core (core 0) controllers for the Pico: the adaptive RBF-NN tracking
3  controller (C3) and the frozen ADP optimal-regulation policy (C4).
4
5  All vector/matrix work uses ulab.numpy (the RBF feature vector and its inner
6  products are vectorised, not Python loops -- C5). The same code runs on a desktop
7  via the NumPy fallback so the Pico results can be checked against the computer
8  results under matching conditions.
9  """
10 try:
11     from ulab import numpy as np                # on the Pico
12     _ON_PICO = True
13 except ImportError:
14     import numpy as np                        # on a desktop (validation)
15     _ON_PICO = False
16
17 import math
18 import params as PA
19
20
21 def _sat(u, umax):
22     return umax if u > umax else (-umax if u < -umax else u)
23
24
25 def y_ref(t):
26     """(y_d, y_d', y_d'') for y_d = A sin(w t)."""
27     A, w = PA.REF_A, PA.REF_W
28     s, c = math.sin(w * t), math.cos(w * t)
29     return A * s, A * w * c, -A * w * w * s, -A * w * w * w * c
30
31
32 # -----
33 # Branch A -- adaptive RBF-NN tracking controller
34 # -----
35 class RBFFController:
36     def __init__(self):
37         b = PA.RBF_BOX
38         g = PA.RBF_GRID
39         ax = [self._lin(b[i][0], b[i][1], g[i]) for i in range(3)]
40         c0, c1, c2 = [], [], []
41         for v0 in ax[0]:
42             for v1 in ax[1]:
43                 for v2 in ax[2]:                # 'ij' ravel order (matches desktop)
44                     c0.append(v0); c1.append(v1); c2.append(v2)
45         self.C0 = np.array(c0); self.C1 = np.array(c1); self.C2 = np.array(c2)
46         self.n = len(c0)
47         self.ih0 = 2.0 / (b[0][1] - b[0][0])
48         self.ih1 = 2.0 / (b[1][1] - b[1][0])
49         self.ih2 = 2.0 / (b[2][1] - b[2][0])
50         self.i2w2 = 1.0 / (2.0 * PA.RBF_WIDTH * PA.RBF_WIDTH)
51         # pre-scaled PER-AXIS centers: d_i = x_i*m_i - A_i gives
52         # (x_i-c_i)*ih_i*sqrt(i2w2) in one small (grid-length) vector op, so
53         # exp(-(d0^2+d1^2+d2^2)) needs no further scaling
54         s = math.sqrt(self.i2w2)
55         self.m0 = self.ih0 * s; self.A0 = np.array(ax[0]) * self.m0
56         self.m1 = self.ih1 * s; self.A1 = np.array(ax[1]) * self.m1
57         self.m2 = self.ih2 * s; self.A2 = np.array(ax[2]) * self.m2
58         self.g0, self.g1, self.g2 = g[0], g[1], g[2]
59         self.W = np.zeros(self.n)
60         self.l2, self.l1, self.l0 = PA.LAM

```

```

61         self.bp0, self.bp1, self.bp2 = PA.BP
62
63     @staticmethod
64     def _lin(a, b, n):
65         return [a] if n == 1 else [a + (b - a) * i / (n - 1) for i in range(n)]
66
67     def _gauss(self, x):
68         """Unnormalized Gaussian vector g(x), exploiting the SEPARABILITY of the
69         grid: exp(-(d0^2+d1^2+d2^2)) over the 6x6x6 Cartesian product equals the
70         outer product of three per-axis 6-element Gaussians. All exp() calls act
71         on grid-length vectors; only two broadcast multiplications touch the full
72         216-element vector -- on the RP2040 this cuts the update from ~6 ms to
73         ~2.5 ms. The (g0*g1)*g2 broadcast order reproduces the 'ij' ravel order
74         of the flat center list, so W indexing is unchanged."""
75         d0 = x[0] * self.m0 - self.A0
76         d1 = x[1] * self.m1 - self.A1
77         d2 = x[2] * self.m2 - self.A2
78         d0 *= d0; d1 *= d1; d2 *= d2
79         d0 *= -1.0; d1 *= -1.0; d2 *= -1.0
80         e0 = np.exp(d0); e1 = np.exp(d1); e2 = np.exp(d2)
81         a = e0.reshape((self.g0, 1)) * e1.reshape((1, self.g1))
82         b = a.reshape((self.g0 * self.g1, 1)) * e2.reshape((1, self.g2))
83         return b.reshape((self.n,))
84
85     def features(self, x):
86         """Normalized Gaussian RBF vector phi(x) (vectorised ulab op)."""
87         g = self._gauss(x)
88         return g / np.sum(g)
89
90     def update(self, x, t, dt):
91         """One controller update: read state x at time t, return u; adapt weights.
92         dt is the controller sampling period Ts (forward-Euler weight update).
93         The normalization phi = g/sum(g) is folded into the two scalar results
94         (W.phi = W.g/sum(g); W-update scaled by 1/sum(g)), saving a 216-element
95         vector division per update on the Pico."""
96         yd, yd1, yd2, yd3 = y_ref(t)
97         e = x[0] - yd; ed = x[1] - yd1; edd = x[2] - yd2
98         g = self._gauss(x)
99         isg = 1.0 / np.sum(g)
100         Dhat = np.dot(self.W, g) * isg
101         u = _sat((Dhat + yd3 - self.l2 * edd - self.l1 * ed - self.l0 * e) / PA.C_IN, PA.
102         U_MAX)
103         # Lyapunov gradient weight update + projection (forward Euler, in place)
104         bPxi = self.bp0 * e + self.bp1 * ed + self.bp2 * edd
105         g *= dt * PA.GAMMA_ADAPT * bPxi * isg
106         self.W -= g
107         self.W = np.clip(self.W, -PA.W_MAX, PA.W_MAX) # projection
108         return u
109
110 # -----
111 # Branch B -- frozen ADP optimal-regulation policy u = -(1/2R) zeta(x)^T w
112 # -----
113 class ADPController:
114     def __init__(self):
115         self.w = np.array(PA.ADP_W)
116         self.mon = PA.ADP_MON
117         self.k = -1.0 / (2.0 * PA.ADP_R)
118
119     def zeta(self, x):
120         """c * d/dx3 of the value-function basis [quadratic ; nonlinear]."""
121         x1, x2, x3 = x[0], x[1], x[2]
122         z = [0.0, 0.0, x1, 0.0, x2, 2.0 * x3] # d/dx3 of quadratic monomials
123         for e in self.mon: # d/dx3 of nonlinear monomials
124             if e[2] == 0:
125                 z.append(0.0)
126             else:
127                 z.append(e[2] * x1 ** e[0] * x2 ** e[1] * x3 ** (e[2] - 1))
128         return np.array(z) * PA.C_IN
129
130     def control(self, x):
131         return _sat(self.k * np.dot(self.zeta(x), self.w), PA.U_MAX)

```

A.6 Pico – two-core real-time orchestration main.py

```

1  """
2  Raspberry Pi Pico top-level (Part 4).  Runs the complete two-core real-time scheme:
3
4      core 1 (second core) : simulates the nonlinear process.  A paced real-time
5                          loop performs ONE scalar RK4 step every dt with the
6                          controller input u held constant (ZOH) and catch-up if
7                          a step is transiently delayed.  (A machine.Timer is
8                          NOT used here: on the RP2040 port soft-timer callbacks
9                          are always dispatched on core 0 -- verified on hardware
10                         via the SIO CPUID register -- so a paced loop inside
11                         the _thread task is what actually runs on core 1.)
12      core 0 (main core)   : a hardware Timer with period Ts = N*dt triggers ONE
13                          controller update -- it reads the latest full state,
14                          computes u, (adapts the RBF weights,) and publishes u
15                          to core 1.  The per-update execution time is measured
16                          and its average and maximum are reported (must stay
17                          below Ts).  gc.collect() runs every GC_EVERY ticks so
18                          collection happens at a controlled point.
19
20  Set MODE = "rbf" (adaptive tracking, C3) or "adp" (frozen optimal policy, C4).
21
22  This module targets MicroPython on the Pico (machine, _thread, ulab).  It is not run
23  on the desktop; the numerical logic is validated off-board by desktop_sim.py, which
24  imports the same params/plant_core1/pico_control unchanged.
25  """
26  import _thread
27  import gc
28  import utime
29  import machine
30  from machine import Timer
31
32  import params as PA
33  import plant_core1 as PL
34  import pico_control as CT
35
36  machine.freq(200_000_000)          # 200 MHz, the officially supported RP2040 fast clock
37
38  MODE = "rbf"                      # "rbf" (Branch A tracking) or "adp" (Branch B regulation
39                                     )
40  X0 = [0.0, 0.0, 0.0] if MODE == "rbf" else [0.7, 0.5, -1.0]
41  GC_EVERY = 10                     # controller ticks between deterministic collections
42
43  # ---- shared state between the cores (single-writer each; lock guards exchange) ----
44  _s1, _s2, _s3 = X0                # state scalars, written by core 1, read by core 0
45  _u = 0.0                          # written by core 0, read by core 1
46  _lock = _thread.allocate_lock()
47  _go = False                        # plant holds X0 until the first u is published
48  _t0 = utime.ticks_us()
49
50  # ---- execution-time statistics (C5) ----
51  _exec_sum = 0
52  _exec_max = 0
53  _exec_cnt = 0
54
55  # -----
56  # core 1: process simulation, one scalar RK4 step every dt (paced loop)
57  # -----
58  def core1_task():
59      global _s1, _s2, _s3
60      dt = PA.PICO_DT
61      dt_us = int(dt * 1e6)
62      x1, x2, x3 = _s1, _s2, _s3
63      while not _go:                  # aligned start: wait for u(X0)
64          pass
65      nxt = utime.ticks_add(utime.ticks_us(), dt_us)
66      while True:
67          if utime.ticks_diff(utime.ticks_us(), nxt) < 0:
68              continue                # busy-wait: the core is dedicated
69          nxt = utime.ticks_add(nxt, dt_us)
70          with _lock:

```

```

71     u = _u
72     x1, x2, x3 = PL.rk4_scalars(x1, x2, x3, u, dt)
73     with _lock:
74         _s1 = x1; _s2 = x2; _s3 = x3
75
76
77 # -----
78 # core 0: controller update, one per timer tick (period Ts = N*dt)
79 # -----
80 def make_controller_tick(ctrl):
81     box = {"k": 0}
82
83     def _tick(timer):
84         global _u, _go, _exec_sum, _exec_max, _exec_cnt
85         with _lock:
86             x = (_s1, _s2, _s3)
87             t = utime.ticks_diff(utime.ticks_us(), _t0) * 1e-6
88             ta = utime.ticks_us()
89             if MODE == "rbf":
90                 u = ctrl.update(x, t, PA.PICO_TS)
91             else:
92                 u = ctrl.control(x)
93             dt_us = utime.ticks_diff(utime.ticks_us(), ta)
94             with _lock:
95                 _u = u
96                 _go = True
97                 k = box["k"]
98                 if k % GC_EVERY == 0:
99                     gc.collect() # deterministic collection, never mid-update
100                 box["k"] = k + 1
101                 _exec_sum += dt_us
102                 _exec_cnt += 1
103                 if dt_us > _exec_max:
104                     _exec_max = dt_us
105     return _tick
106
107
108 def main():
109     ctrl = CT.RBFCtrlController() if MODE == "rbf" else CT.ADPCtrlController()
110     _thread.start_new_thread(core1_task, ()) # plant on core 1
111     utime.sleep_ms(50) # let core 1 start
112     Timer(period=int(PA.PICO_TS * 1000), mode=Timer.PERIODIC,
113           callback=make_controller_tick(ctrl)) # controller on core 0
114     # report loop
115     while True:
116         utime.sleep_ms(1000)
117         with _lock:
118             y = _s1
119             avg = (_exec_sum / _exec_cnt) if _exec_cnt else 0
120             print("MODE=%s y=%.4f u=%.3f exec avg=%.1f us max=%d us (Ts=%d us)"
121                   % (MODE, y, _u, avg, _exec_max, int(PA.PICO_TS * 1e6)))
122
123
124 if __name__ == "__main__":
125     main()

```

A.7 Pico – logged hardware experiment pico_experiment.py

```

1 """
2 Bounded, logged run of the two-core real-time scheme (Part 4 hardware experiment).
3
4 Core allocation (same intent as main.py):
5
6     core 1 : the nonlinear process. A paced real-time loop performs one scalar
7             RK4 step (plant_core1.rk4_scalars) every dt, with catch-up if a step
8             is transiently delayed (e.g. by a GC pause on the other core). A
9             plain machine.Timer is NOT used for the plant: on the RP2040 port
10             MicroPython soft-timer callbacks are always dispatched on core 0
11             (verified on hardware via the SIO CPUID register), so a paced loop
12             inside the _thread task is the only way to actually run the process
13             on the second core.

```

```

14     core 0 : the controller, triggered by a hardware Timer every Ts = N*dt. It
15             copies the three state scalars under the lock, computes u, publishes
16             it, and logs (y, u, exec time). gc.collect() runs every GC_EVERY
17             ticks so collection happens at a controlled point, never mid-update.
18
19 The run lasts a fixed simulated time T; the log is printed as CSV over USB so the
20 desktop side (plot_hw_results.py) can draw the Pico-vs-computer overlays and the
21 timing histogram from real on-chip data.
22
23 Usage from the host:
24
25     mpremote exec "import pico_experiment; pico_experiment.run('rbf', 20.0)"
26     mpremote exec "import pico_experiment; pico_experiment.run('adp', 8.0)"
27
28 Each call shuts the timer down and releases the second core, so both runs can be
29 captured in one session.
30 """
31 import _thread
32 import array
33 import gc
34 import machine
35 import utime
36 from machine import Timer
37
38 import params as PA
39 import plant_core1 as PL
40 import pico_control as CT
41
42 machine.freq(200_000_000)      # officially supported RP2040 fast clock
43
44 _CPUID = 0xD0000000            # SIO CPUID register: 0 on core 0, 1 on core 1
45 GC_EVERY = 10                 # controller ticks between deterministic collections
46
47 # shared between the cores (single-writer each; lock guards the exchange)
48 _s1 = _s2 = _s3 = 0.0         # state scalars, written by core 1
49 _u = 0.0                      # input, written by core 0
50 _lock = _thread.allocate_lock()
51 _plant_ticks = 0
52 _plant_cpu = -1
53 _plant_late_max = 0           # worst plant-step start delay [us]
54 _core1_stop = False
55 _core1_done = False
56 _go = False                   # plant holds x0 until the first u is published
57
58
59 def _core1_task():
60     """Plant real-time loop on the second core: one RK4 step every dt."""
61     global _s1, _s2, _s3, _plant_ticks, _plant_cpu, _core1_done, _plant_late_max
62     _plant_cpu = machine.mem32[_CPUID] & 1
63     dt = PA.PICO_DT
64     dt_us = int(dt * 1e6)
65     x1, x2, x3 = _s1, _s2, _s3
66     while not _go and not _core1_stop:      # aligned start: wait for u(x0)
67         pass
68     nxt = utime.ticks_add(utime.ticks_us(), dt_us)
69     while not _core1_stop:
70         late = utime.ticks_diff(utime.ticks_us(), nxt)
71         if late < 0:
72             continue                        # busy-wait: the core is dedicated
73         if late > _plant_late_max:
74             _plant_late_max = late
75         nxt = utime.ticks_add(nxt, dt_us)
76         with _lock:
77             u = _u
78             x1, x2, x3 = PL.rk4_scalars(x1, x2, x3, u, dt)
79         with _lock:
80             _s1 = x1; _s2 = x2; _s3 = x3
81             _plant_ticks += 1
82         _core1_done = True
83
84
85 def run(mode="rbf", T=20.0):

```

```

86     global _s1, _s2, _s3, _u, _core1_stop, _core1_done, _plant_ticks, _plant_late_max,
87         _go
88     K = int(round(T / PA.PICO_TS))
89     Ts = PA.PICO_TS
90
91     y_log = array.array("f", bytearray(4 * K))
92     u_log = array.array("f", bytearray(4 * K))
93     exec_log = array.array("H", bytearray(2 * K)) # controller update [us]
94     tot_log = array.array("H", bytearray(2 * K)) # full tick incl. log/gc [us]
95
96     ctrl = CT.RBFCController() if mode == "rbf" else CT.ADPCController()
97     _s1, _s2, _s3 = (0.0, 0.0, 0.0) if mode == "rbf" else (0.7, 0.5, -1.0)
98     _u = 0.0
99     _plant_ticks = 0
100    _plant_late_max = 0
101    _core1_stop = False
102    _core1_done = False
103    _go = False
104
105    box = {"k": 0, "cpu": -1}
106
107    def _ctrl_tick(timer):
108        global _u, _go
109        k = box["k"]
110        if k >= K:
111            return
112        box["cpu"] = machine.mem32[_CPUID] & 1
113        ta = utime.ticks_us()
114        with _lock:
115            x = (_s1, _s2, _s3)
116            if mode == "rbf":
117                u = ctrl.update(x, k * Ts, Ts)
118            else:
119                u = ctrl.control(x)
120            te = utime.ticks_diff(utime.ticks_us(), ta)
121            with _lock:
122                _u = u
123                _go = True # release the plant on the first published u
124                y_log[k] = x[0]
125                u_log[k] = u
126                if k % GC_EVERY == 0:
127                    gc.collect() # collection at a controlled point, never mid-update
128                    tt = utime.ticks_diff(utime.ticks_us(), ta)
129                    exec_log[k] = te if te < 65535 else 65535
130                    tot_log[k] = tt if tt < 65535 else 65535
131                    box["k"] = k + 1
132
133    gc.collect()
134    _thread.start_new_thread(_core1_task, ())
135    utime.sleep_ms(100) # let the plant loop start
136    tim0 = Timer(period=int(Ts * 1000), mode=Timer.PERIODIC, callback=_ctrl_tick)
137
138    t_wall = utime.ticks_ms()
139    while box["k"] < K:
140        utime.sleep_ms(50)
141        wall = utime.ticks_diff(utime.ticks_ms(), t_wall)
142
143    tim0.deinit()
144    _core1_stop = True
145    while not _core1_done:
146        utime.sleep_ms(5)
147
148    ex = exec_log[1:] # skip the cold first tick
149    print("# mode=%s T=%.1f Ts=%g dt=%g K=%d plant_ticks=%d wall_ms=%d plant_late_max_us=%d" %
150          (mode, T, Ts, PA.PICO_DT, K, _plant_ticks, wall, _plant_late_max))
151    print("# freq_hz=%d ctrl_cpu=%d plant_cpu=%d micropython=%s" %
152          (machine.freq(), box["cpu"], _plant_cpu,
153           ".".join(str(v) for v in __import__("sys").implementation.version[:3])))
154    print("# exec_us avg=%.1f max=%d | tick_us(incl log/gc) avg=%.1f max=%d | Ts_us=%d" %
155          (sum(ex) / len(ex), max(ex), sum(tot_log[1:]) / (K - 1), max(tot_log[1:]),
156           int(Ts * 1e6)))
157    print("k,y,u,exec_us,tot_us")

```

```
157     for k in range(K):  
158         print("%d,%.6f,%.5f,%d,%d" % (k, y_log[k], u_log[k], exec_log[k], tot_log[k]))  
159     print("# done")
```

B References

References

- [1] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Englewood Cliffs, NJ: Prentice-Hall, 1991.
- [2] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2002.
- [3] F. L. Lewis, D. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed. Hoboken, NJ: Wiley, 2012.
- [4] Y. Jiang and Z.-P. Jiang, “Computational adaptive optimal control for continuous-time linear systems with completely unknown dynamics,” *Automatica*, vol. 48, no. 10, pp. 2699–2704, 2012.
- [5] K. G. Vamvoudakis and F. L. Lewis, “Online actor–critic algorithm to solve the continuous-time infinite horizon optimal control problem,” *Automatica*, vol. 46, no. 5, pp. 878–888, 2010.
- [6] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 3rd ed. Reading, MA: Addison-Wesley, 1998.